

Stacks and Queues

3.2 The Stack Abstract Data type

A stack is a **LIFO**(Last In First Out) list.

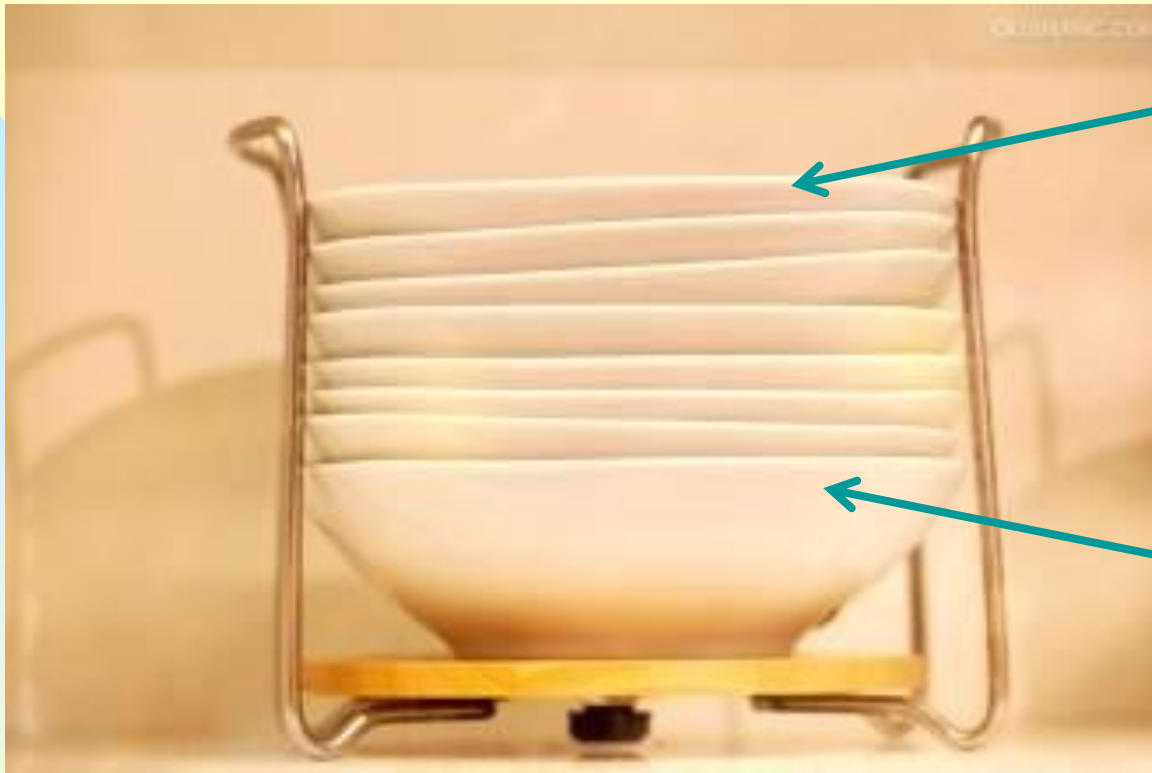
---Ordered (Linear) **list**.

---One end is called **top**.

---The other end is called **bottom**.

---**Pushes**(insertions) and **Pops**(removals) are made at the **top** end only.

Example from life: Stack Of Cups



Top

Bottom

Add a cup to the stack **on the top**.

Remove a cup from the stack **on the top**.

ADT 3.1 Stack

```
template <class T>
class Stack
{ // A finite ordered list with zero or more elements.
public:
    Stack(int stackCapacity = 10);
    //Creates an empty stack with initial capacity of stackCapacity

    bool IsEmpty() const;
    //If number of elements in the stack is 0, true else false

    T& Top() const;
    // Return the top element of stack

    void Push(const T& item);
    // Insert item into the top of the stack
    void Pop();
    // Delete the top element of the stack.
};
```

Represent a stack using array

To implement STACK ADT, we can use

- **an array** to represent the stack
- a variable **top** to point to the stack top, and initially top is set to -1 .

So we have the following data members of Stack:

private:

T* stack;

int top;

int capacity;

```
template <class T>
Stack<T>::Stack(int stackCapacity): capacity(stackCapacity)
{
    if (capacity < 1) throw “Stack capacity must be > 0”;
    stack = new T[capacity];
    top = -1;
}
```

```
template <class T>
inline bool Stack<T>::IsEmpty() const
{
    return(top == -1);
}
```

```
template <class T>
inline T& Stack<T>::Top() const
{
    if (IsEmpty()) throw “Stack is Empty”;
    return stack[top];
}
```

```
template <class T>
void Stack<T>::Push(const T& x)
{
    if (top == capacity - 1) //if the stack is full
    {
        ChangeSize1D(stack, capacity, 2*capacity);
        capacity *= 2;
    }
    stack[++top] = x;
}
```

The template function `ChangeSize1D` changes the size of a 1-Dimensional array of type `T` from `oldSize` to `newSize`:

```
template <class T>
void ChangeSize1D(T* a, const int oldSize, const int newSize)
{
    if (newSize < 0) throw “New length must be >= 0”;
    T* temp = new T[newSize];
    int number = min(oldSize, newSize);
    copy(a, a + number, temp);
    delete [] a;
    a = temp;
}
```

```
template <class T>
void Stack<T>::Pop()
{ // Delete top element of stack.
    if (IsEmpty()) throw “Stack is empty, cannot delete.”;
    stack[top--].~T(); // destructor for T
}
```


3.3 The Queue Abstract Data Type

A queue is a **FIFO(First-In-First-Out)** list

---Linear **list**.

---One end is called **front**.

---The other end is called **rear**.

---Additions are done at the **rear** only.

---Removals are made from the **front** only.

Example from life: Bus Stop Queue

Bus
Stop



front



rear



rear



rear



rear



Example from life: Bus Stop Queue

Bus
Stop



front



rear



Example from life: Bus Stop Queue

Bus
Stop



front



rear



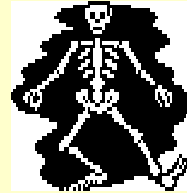
Example from life: Bus Stop Queue



front



rear



rear

ADT 3.2 Queue

```
template <class T>
class Queue
{ // A finite ordered list with zero or more elements.
public:
    Queue (int queueCapacity = 10);
    // Creates an empty queue with initial capacity of
    // queueCapacity

    bool IsEmpty() const;

    T& Front() const; //Return the front element of the queue.

    T& Rear() const; //Return the rear element of the queue.

    void Push(const T& item);
    //Insert item at the rear of the queue.
    void Pop();
    // Delete the front element of the queue.
};
```

Represent a queue using an array

1. Using an **array** `queue[]` with the **front (first)** element of the queue in **`queue[0]`**, and the rear (last) element in **`queue[rear]`**.

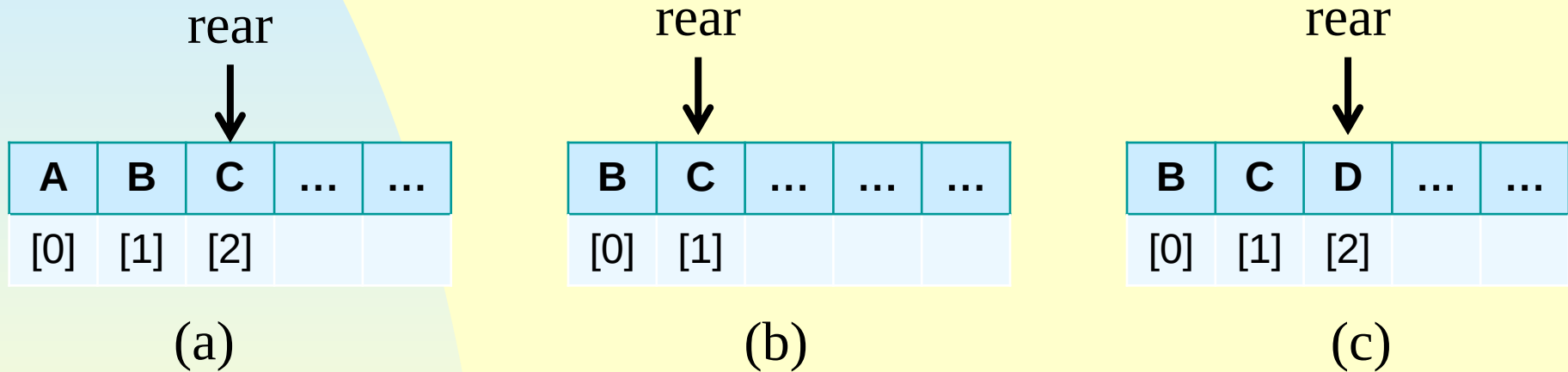


Figure 3.5: Queues represented with **front element in `queue[0]`**

- Push: $O(1)$ if the queue is not full.
- Pop: $O(n)$
- **Problem:** data movement when Pop.

2. Using a variable, **front**, to keep track of the location of the front element. The queue elements are in **queue[front], ..., queue[rear]**.

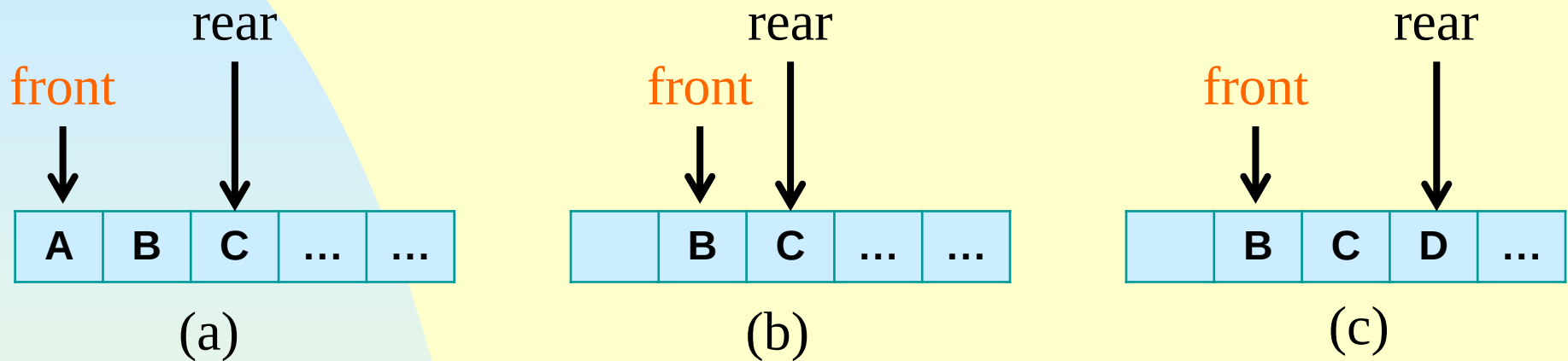
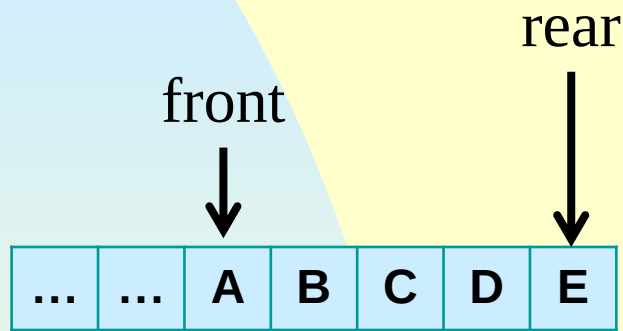


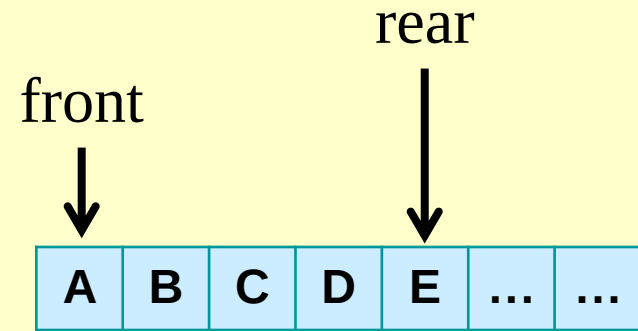
Figure 3.6: Queues represented with front element in queue [0]

- Pop: $O(1)$
- Push: $O(1)$
- **Problem**: When the left end is empty but the right end is full, how to add an element?

- **One possible way** is to shift all elements to the left end of the array if the left end is empty, when the array positions are viewed as arranged in the **straight line**,



(a) Before shift

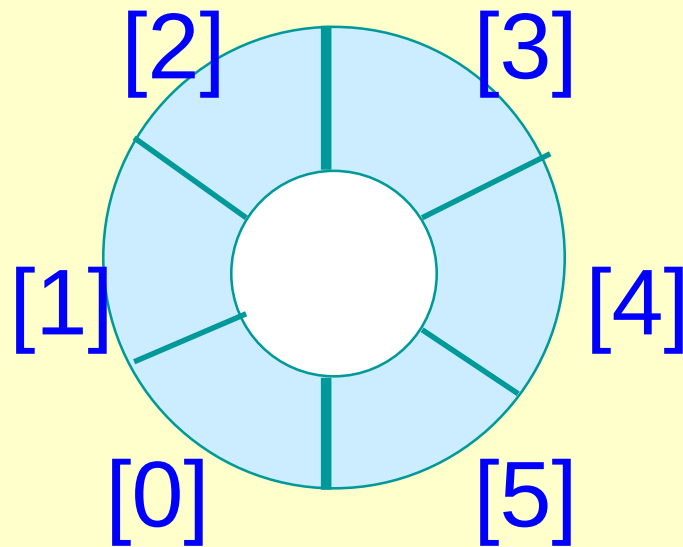


(b) After shift

Figure 3.7: Shifting queue elements to the left

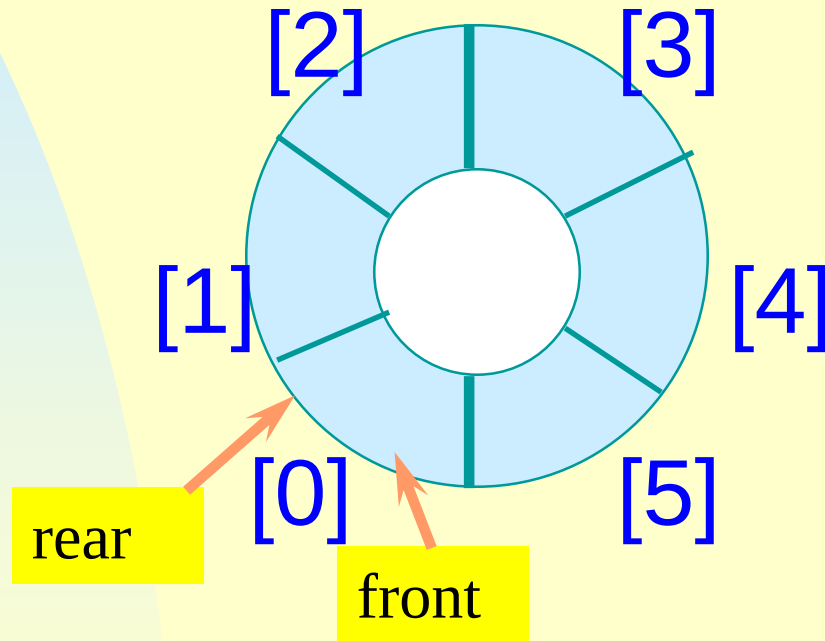
- **Problem:** data movement
- Can we improve it?
- Think of the array positions as arranged in a **circle** rather than in a straight line.

Circular view of array- Circular Queue



Circular Queue Example

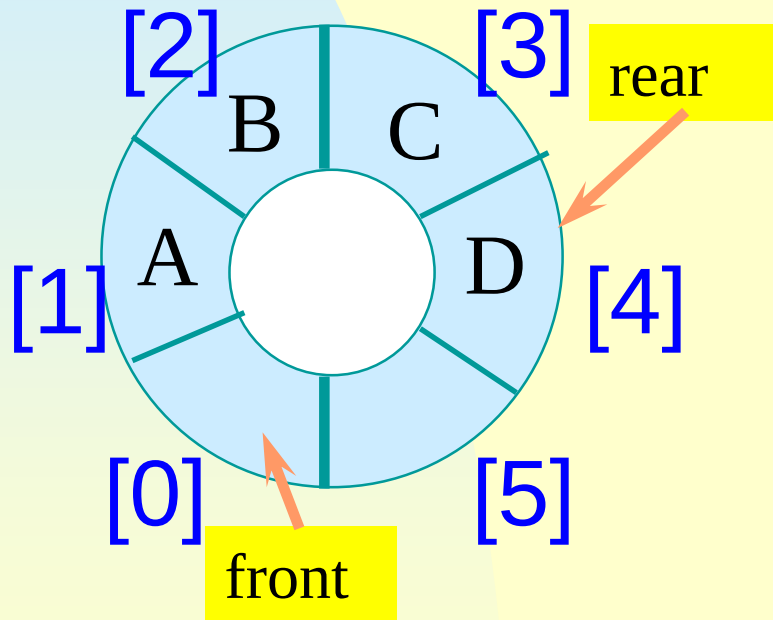
- (a) Initial



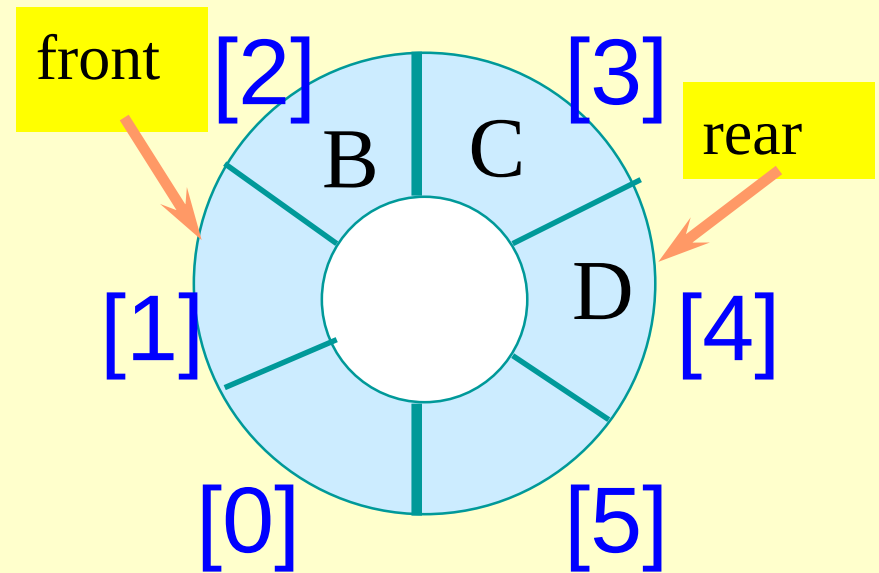
Here, **front** is changed to point one position **counterclockwise** from the first element.

This change simplifies the codes slightly.

- **(b) Addition**



- **(c) Deletion**



Representation of Circular Queue-using array

To implement the **Queue ADT using an array in a circular fashion**, we have the following data members declarations in the Queue class:

private:

```
T* queue;  
int front,  
    rear,  
    capacity;
```

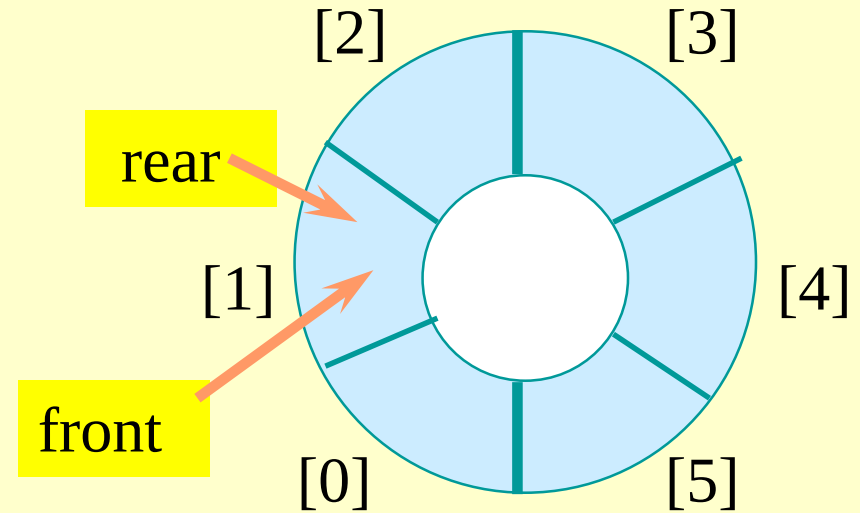
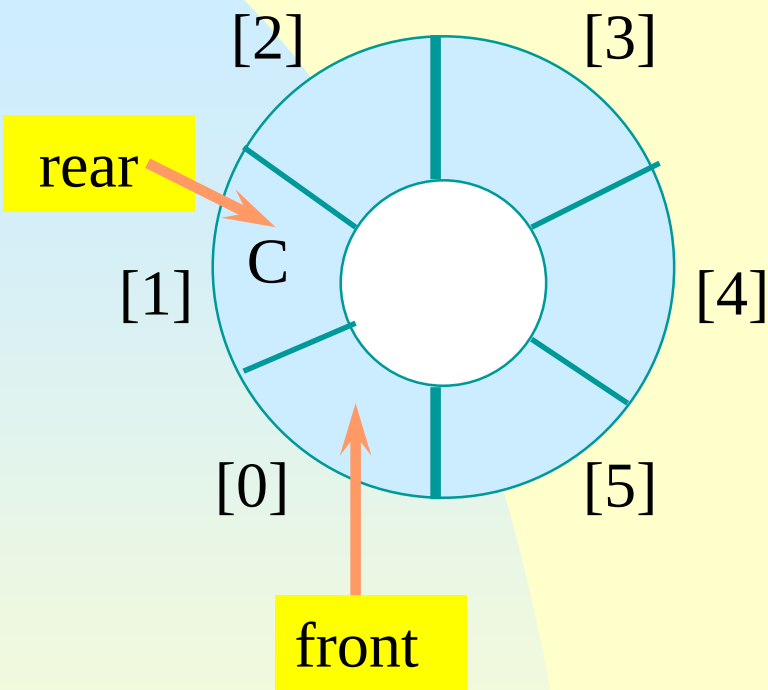
When the **array** is viewed as a **circle**, each array position has a **next** and a **previous position**. The position **next to position capacity-1 is 0**, and the position that **precedes 0 is capacity-1**.

Now we discuss the implementation of operations for circular queue

How does front and rear change when insert or delete an element?

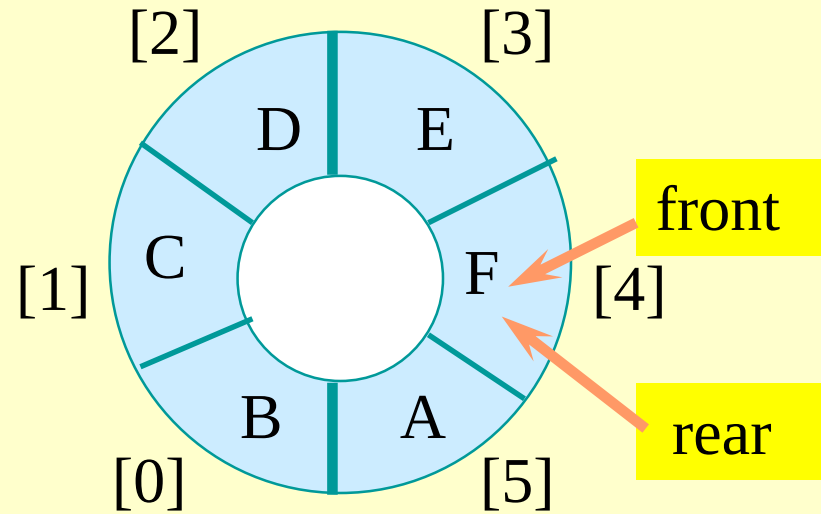
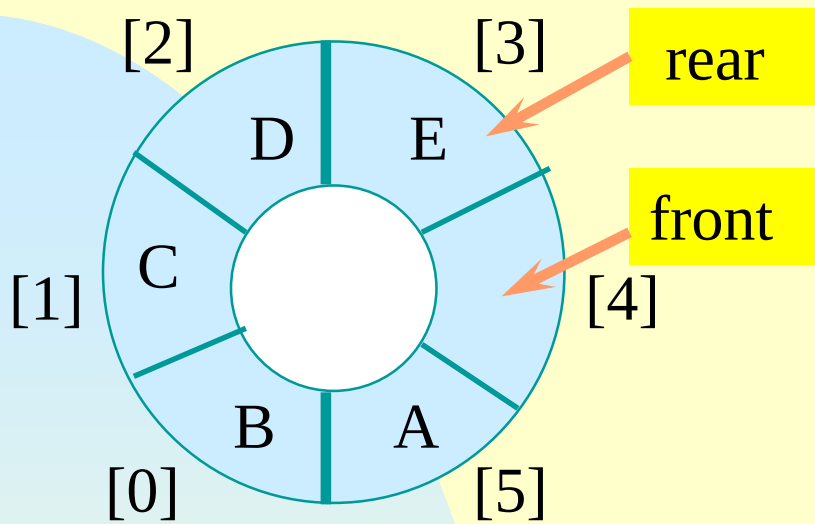
- When the **queue rear is at capacity-1**, after an element is **inserted**, the rear is 0:
 - **if (rear == capacity-1) rear = 0;**
else rear++; **or**
 - **rear = (rear + 1) % capacity;**
- When the **queue front is at capacity-1**, after an element is **popped**, the front is 0:
 - **if (front == capacity-1) front = 0;**
else front++; **or**
 - **front = (front + 1) % capacity;**

Question: How to judge a circular queue is empty or full?



An empty queue

- When a queue is just constructed, it is empty, $\text{front} = \text{rear} = 0$.
- When a series of removes causes the queue to become empty, $\text{front} = \text{rear}$.



An full queue

- When a series of adds causes the queue to become full, **front = rear**.
- So, we cannot distinguish between a full queue and an empty queue!

How to solve the “empty or full” problem

1. Don't let the queue get full.

When the addition of an element will cause the queue to be full, increase array size.

This is what the textbook does.

2. Define a boolean variable `lastOperationIsPut`.

Following each `put` set this variable to `true`.

Following each `remove` set to `false`.

Queue is empty iff `(front == rear)`
`&& !lastOperationIsPut`

Queue is full iff `(front == rear) &&`
`lastOperationIsPut`

3. Define an integer variable `size`.

Following each `put` do `size++`.

Following each `remove` do `size--`.

Queue is empty iff `(size == 0)`

Queue is full iff `(size == capacity)`

Implementation of operations for circular queue (Based on the method that the textbook deals with empty and full queue)

```
template <class T>
Queue<Type>::Queue(int queueCapacity):
    capacity(queueCapacity)
{
    if (capacity < 1) throw "Queue capacity must > 0";
    queue = new T[capacity];
    front = rear = 0;
}
```

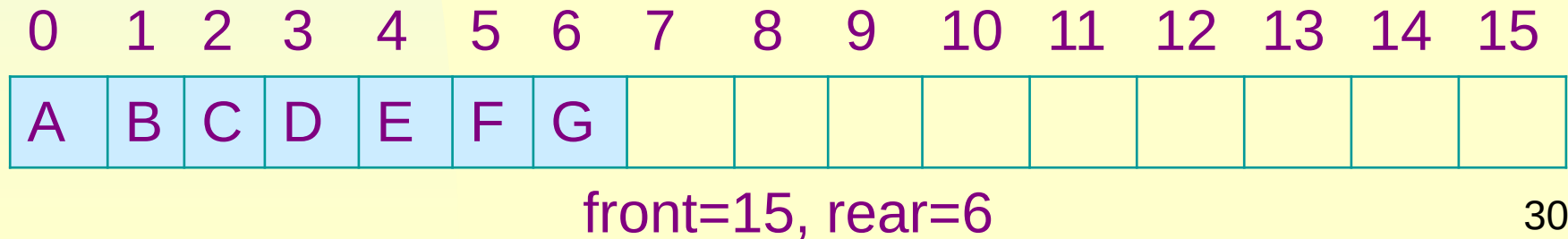
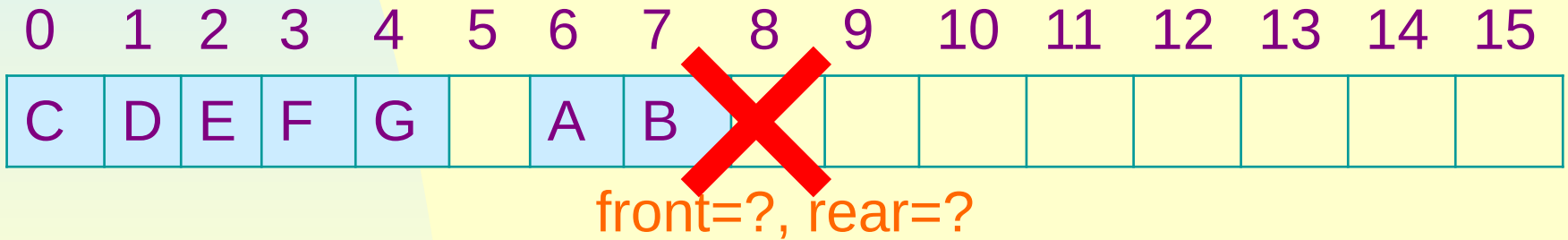
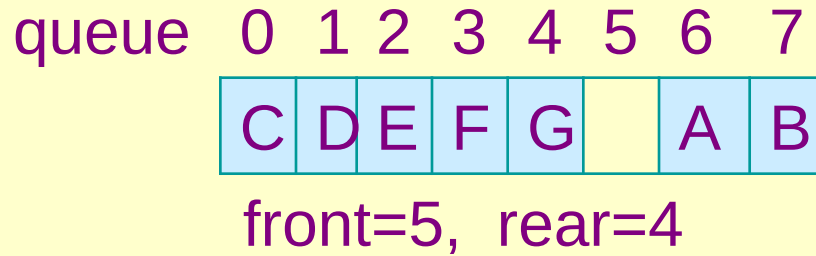
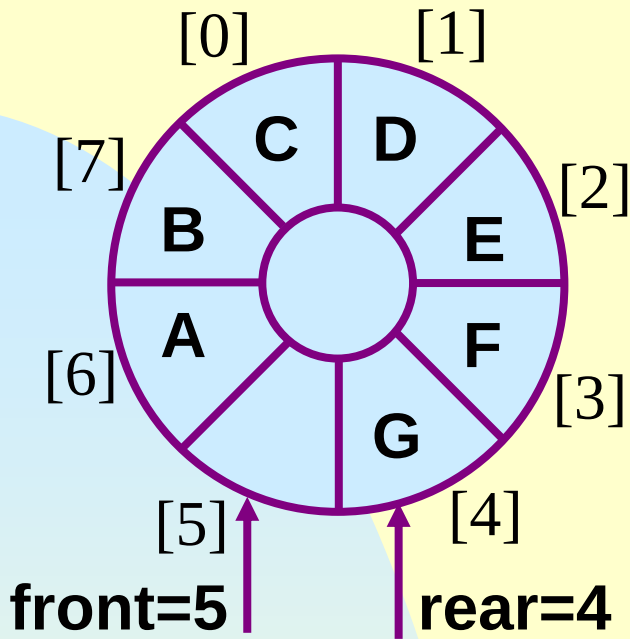
```
template <class T>
Inline bool Queue<T>::IsEmpty()
{ return front==rear };
```

```
template <class T>
inline T& Queue<T>::Front()
{
    if (IsEmpty()) throw “Queue is empty. No front element”;
    return queue[(front+1)%capacity];
}
```

```
template <class T>
inline T& Queue<T>::Rear()
{
    if (IsEmpty()) throw "Queue is empty. No rear element";
    return queue[rear];
}
```

```
template <class T>
void Queue<T>::Push(const T& x)
{ // add x at rear of queue
    if ((rear+1)%capacity == front)
    { // queue full, double capacity
        // code to double queue capacity comes here
    }
    rear = (rear+1)%capacity;
    queue[rear] = x;
}
```

How to double the capacity of queue?



The correct solution is as follows:

- (1) Create **a new array** newQueue with the twice capacity.
- (2) Copy the **second segment** (i.e., the elements queue[front+1] through queue[capacity-1]) to positions in newQueue **beginning at 0**.
- (3) Copy the **first segment** (i.e., the elements queue[0] through queue[rear]) to positions in newQueue beginning at **capacity-front-1**.

The code is in the next slide.

```
// allocate an array with twice the capacity
T* newQueue = new T[2*capacity];

// copy from queue to newQueue
int start = (front+1)%capacity;
if (start < 2) //front==0 or capacity-1, just one segment
    // no wrap around
    copy(queue+start, queue+start+capacity-1, newQueue);
else
{ // queue wraps around, copy the 2nd segment at first, then the 1st one
    copy(queue+start, queue+capacity, newQueue);
    copy(queue, queue+rear+1, newQueue+capacity-start);
}

// switch to newQueue
front = 2*capacity-1; //at the end
rear = capacity-2;
capacity *= 2;
delete [] queue;
queue = newQueue;
```

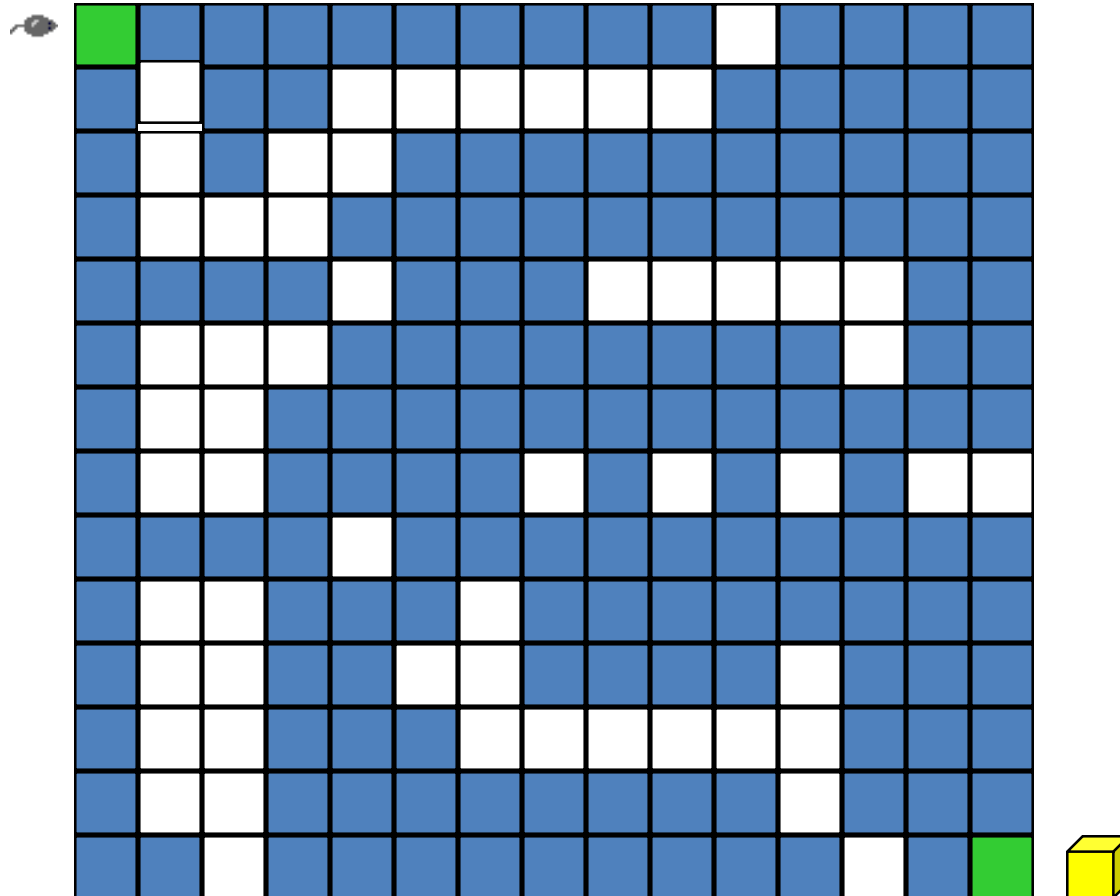


```
template <class T>
void Queue<T>::Pop()
{ // Delete front element from queue
  if (IsEmpty()) throw "Queue is empty. Cannot delete";
  front = (front+1)%capacity;
  queue[front].~T();
}
```

For the circular representation, the worst-case **add and delete** times (assuming no array resizing is needed) are **$O(1)$** .

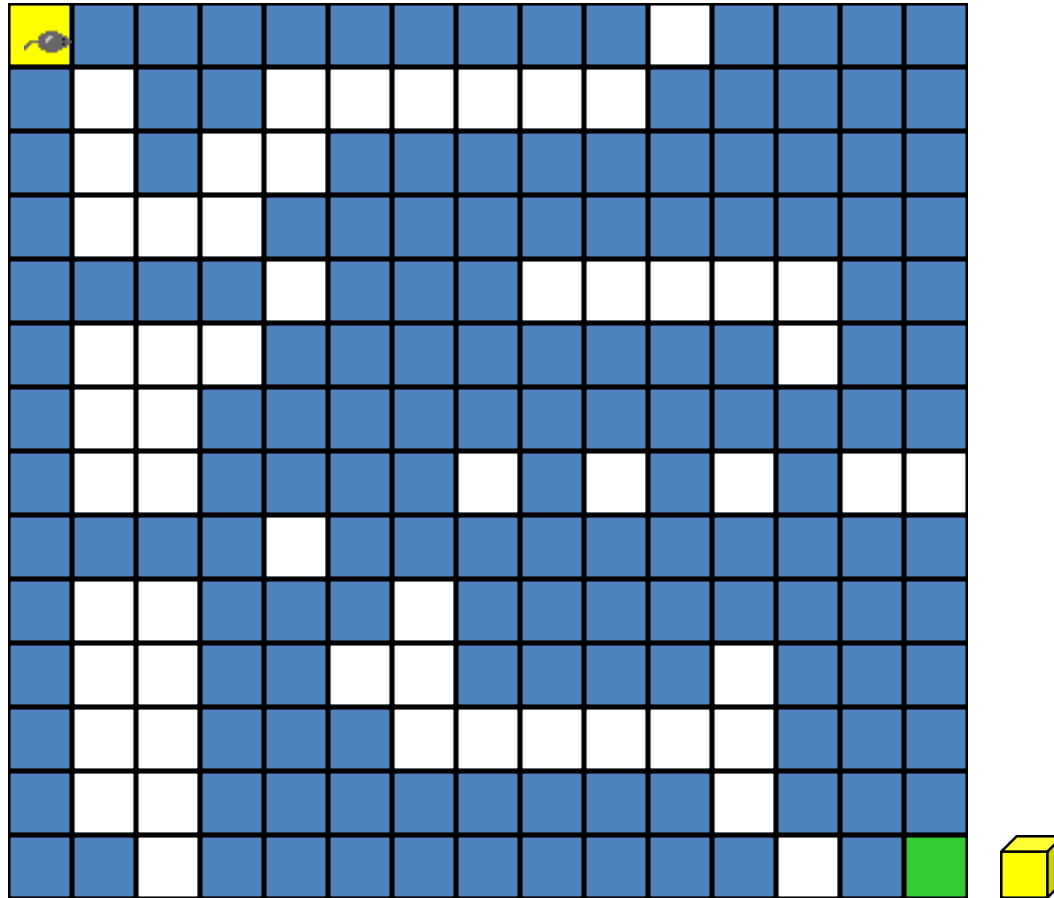
3.5 A Mazing Problem

Rat In A Maze



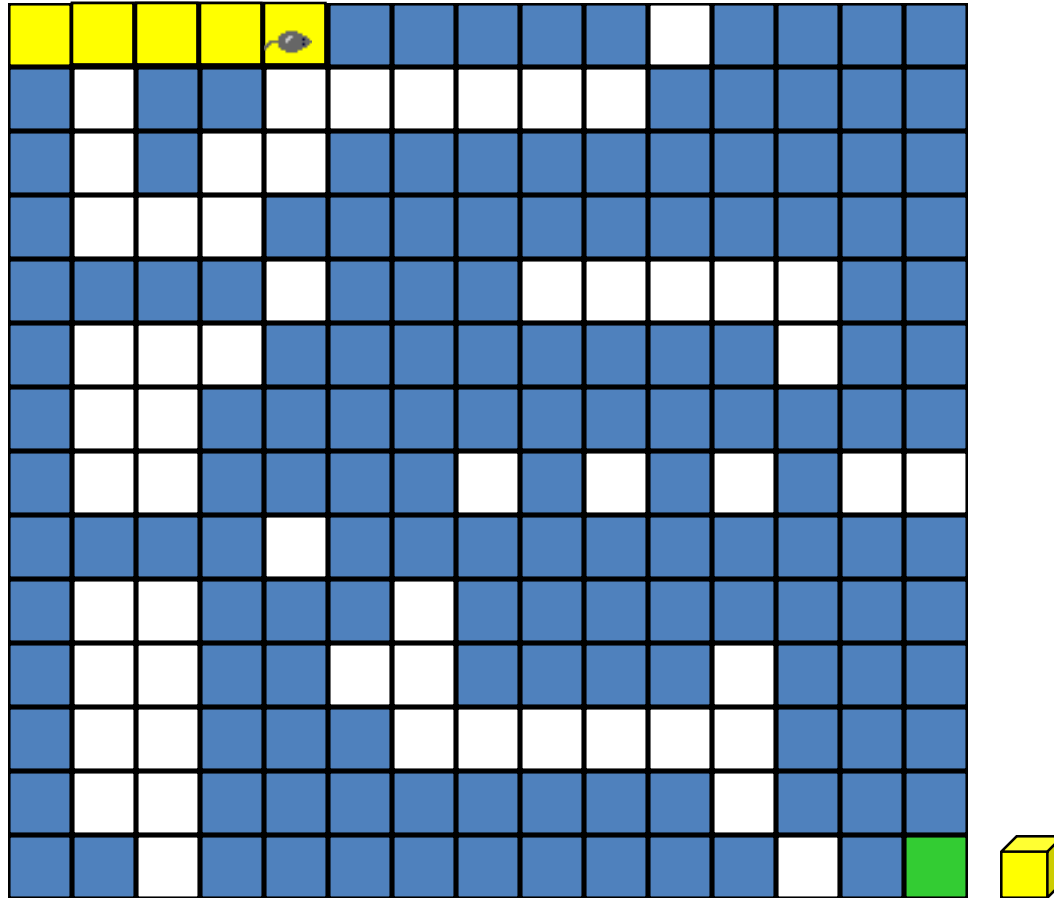
- White squares cannot be moved to.

Rat In A Maze



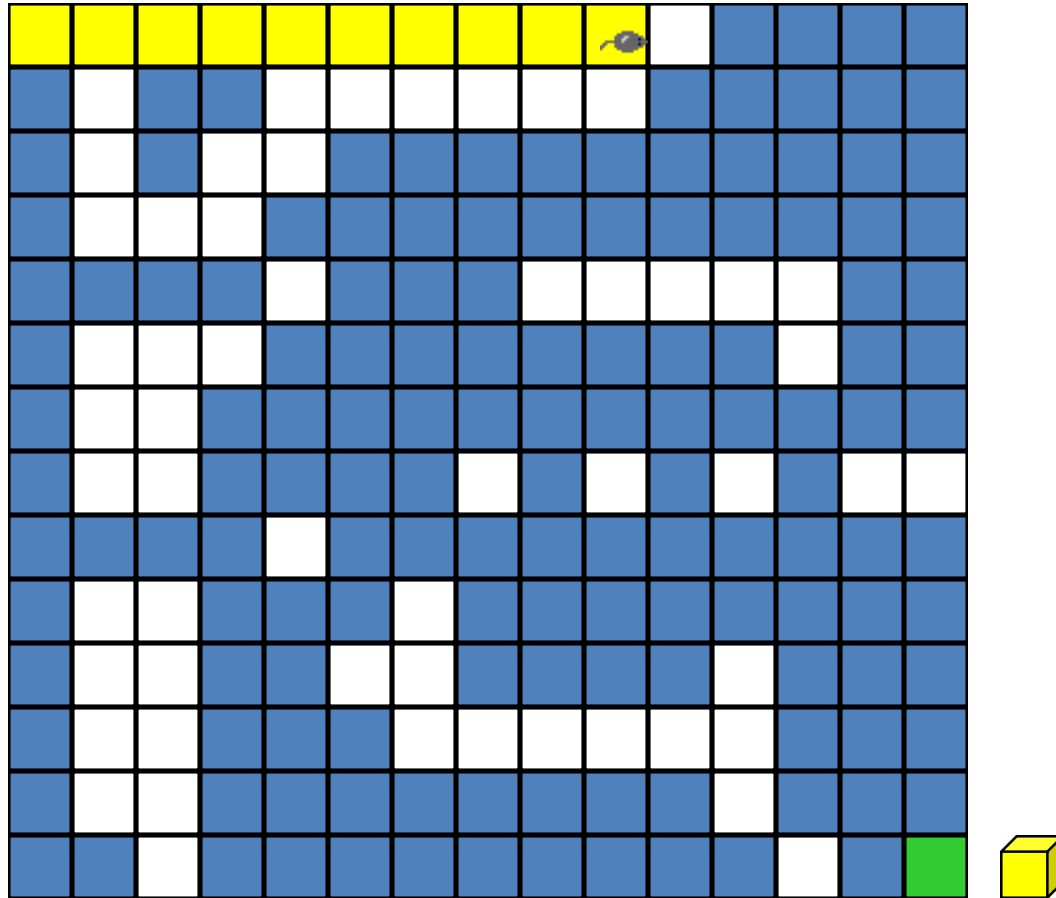
- Move order is: right, down, left, up
- Make marks to avoid revisit

Rat In A Maze



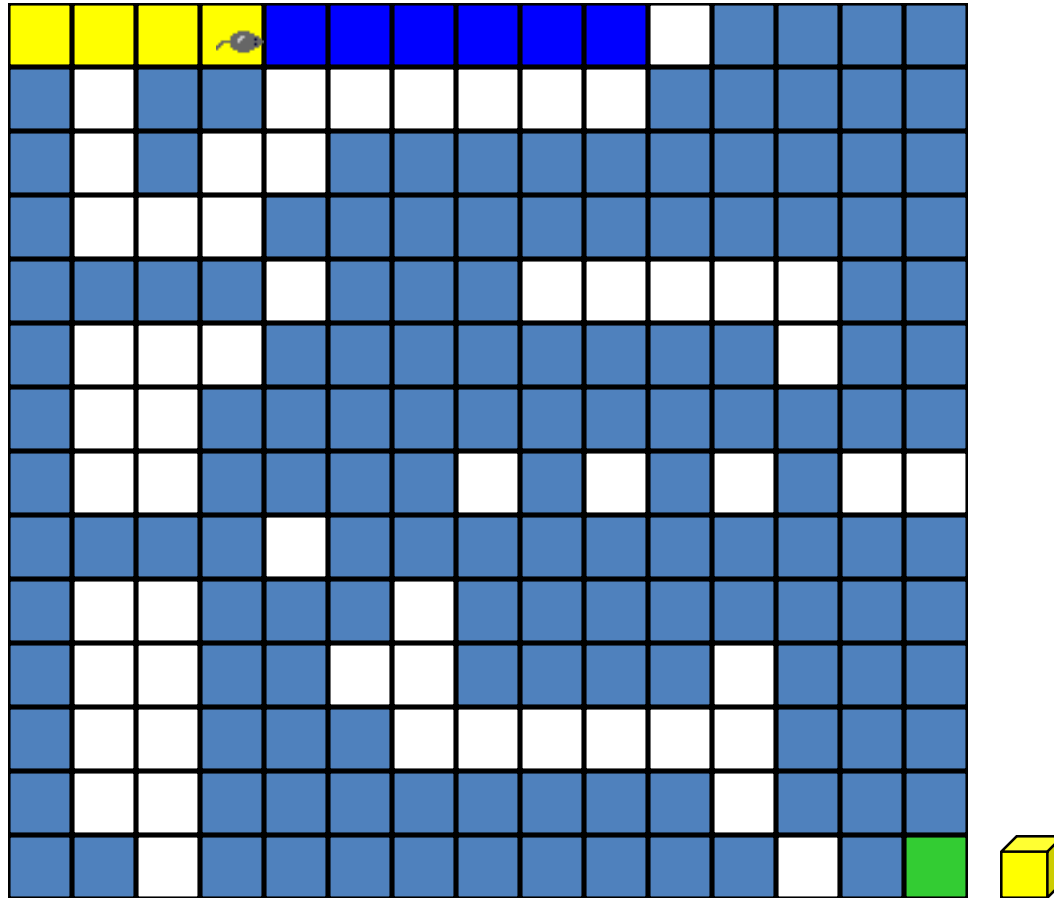
- Moving order is: right, down, left, up
- Make marks to avoid revisit.

Rat In A Maze



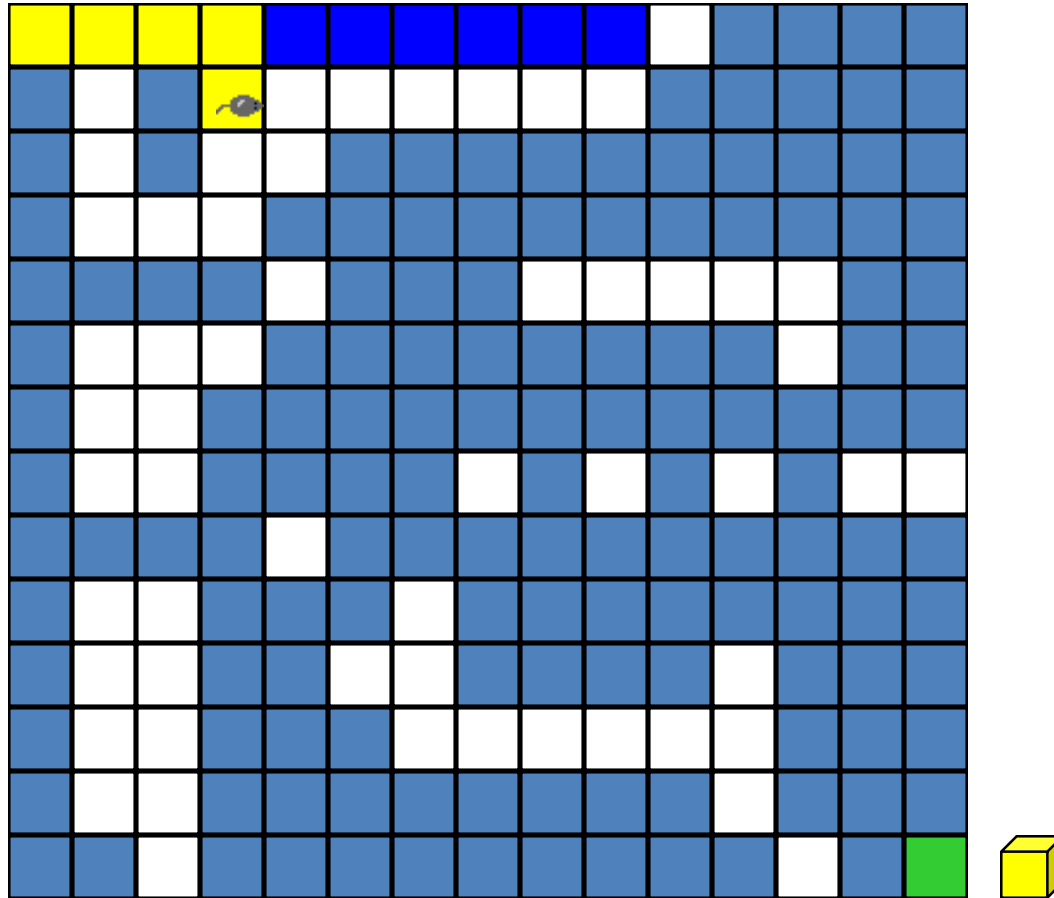
- Try another direction. If all directions are tried, the rat **moves backward** until it reaches a square from which a forward move is possible.

Rat In A Maze



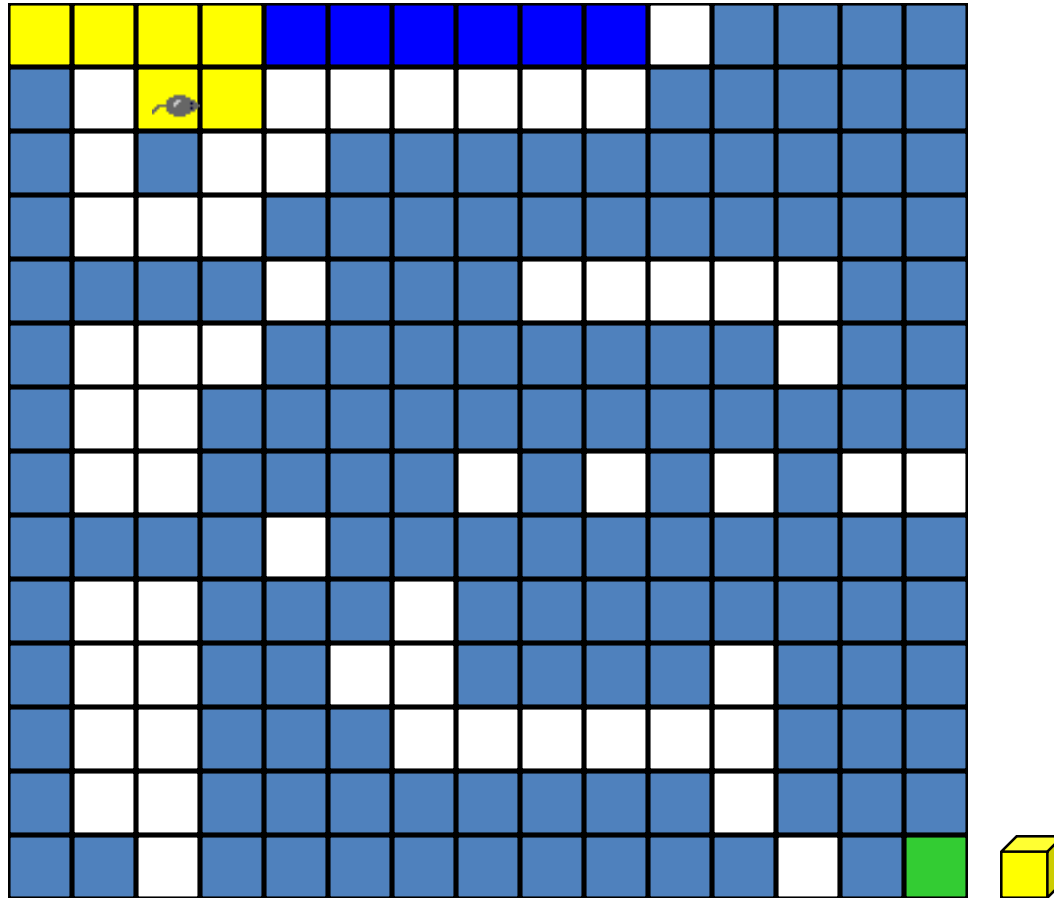
- Move down.

Rat In A Maze



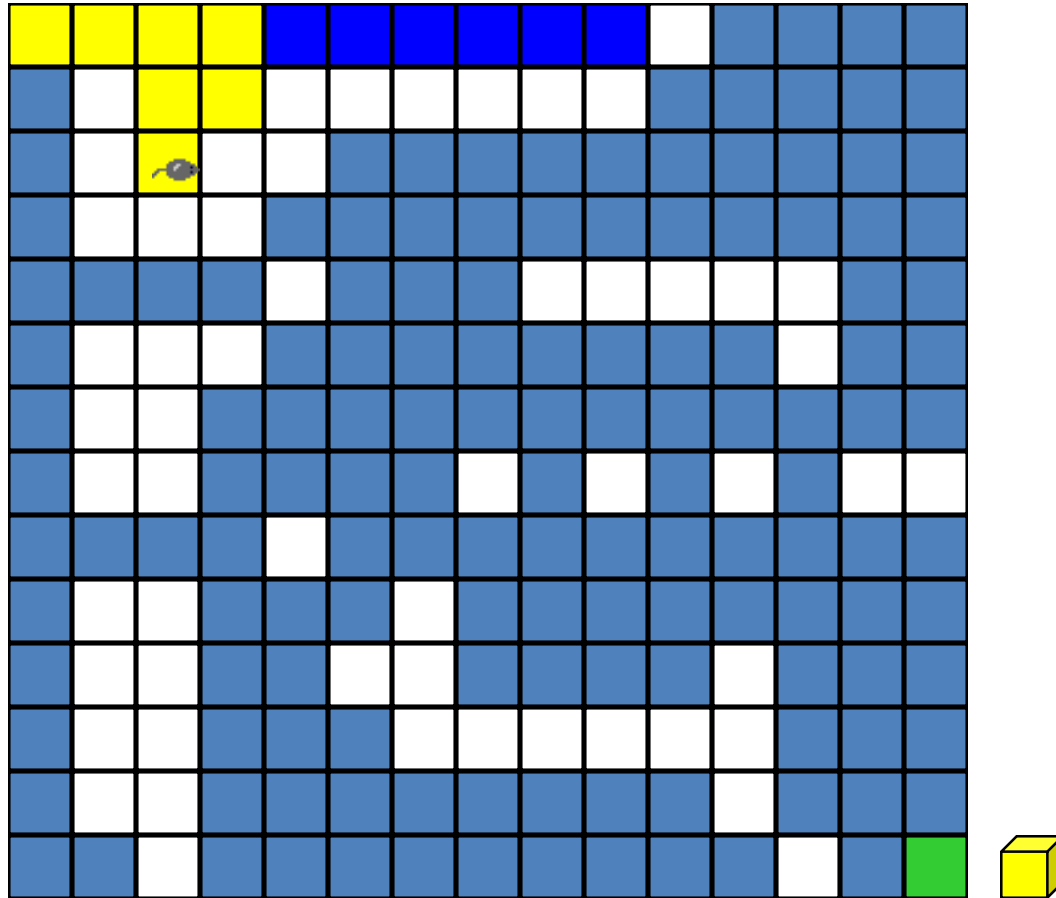
- Move left.

Rat In A Maze



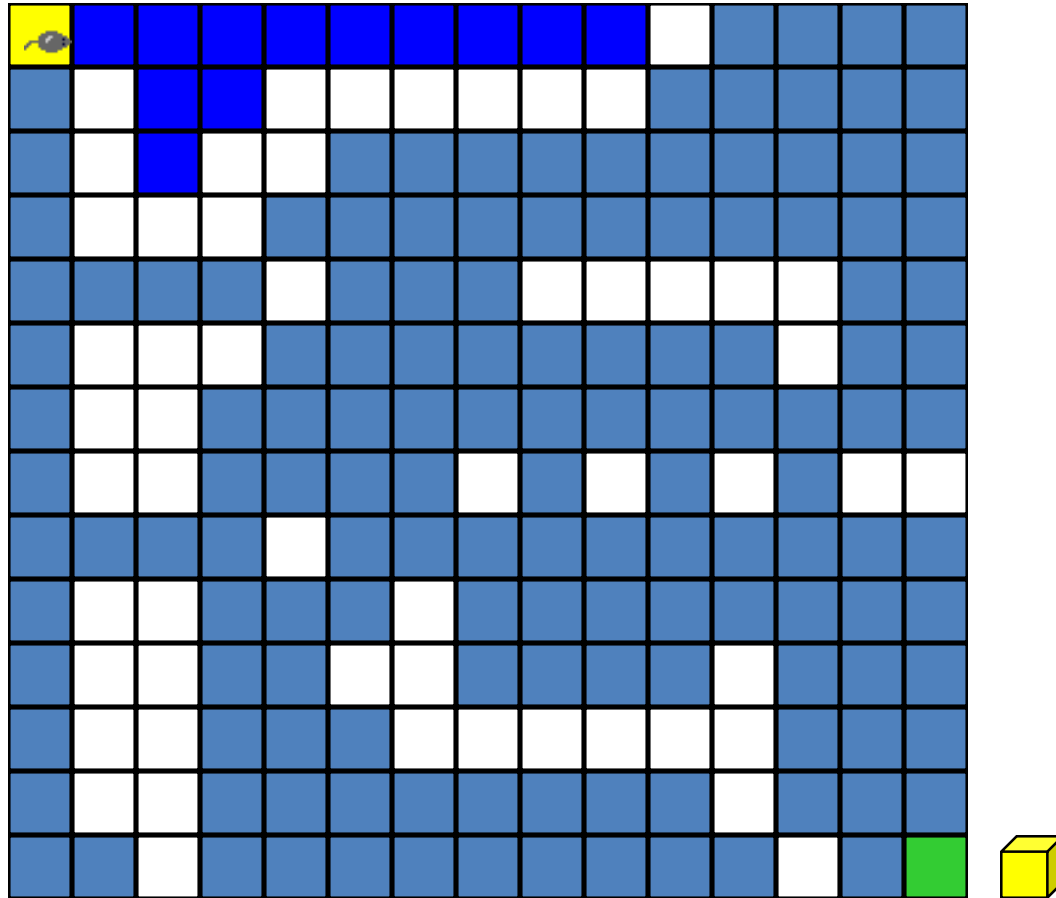
- Move down.

Rat In A Maze



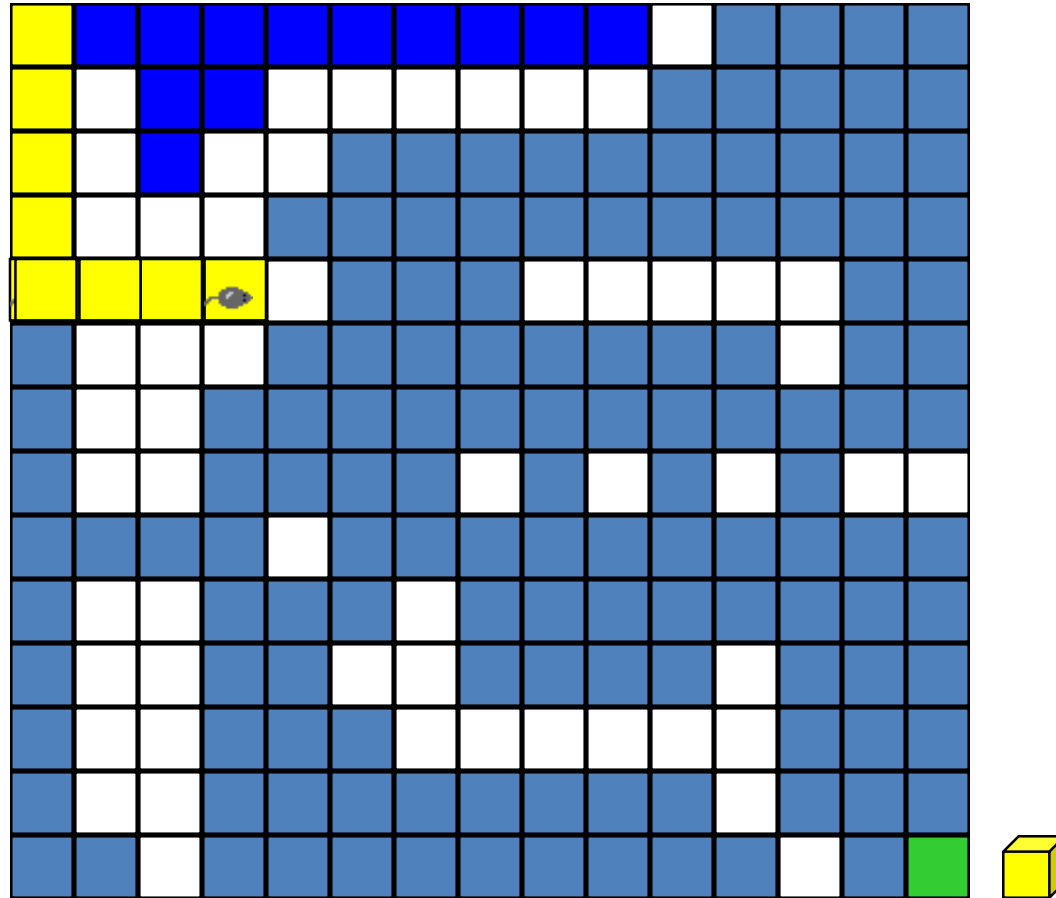
- Move backward until it reaches a square from which a forward move is possible.

Rat In A Maze



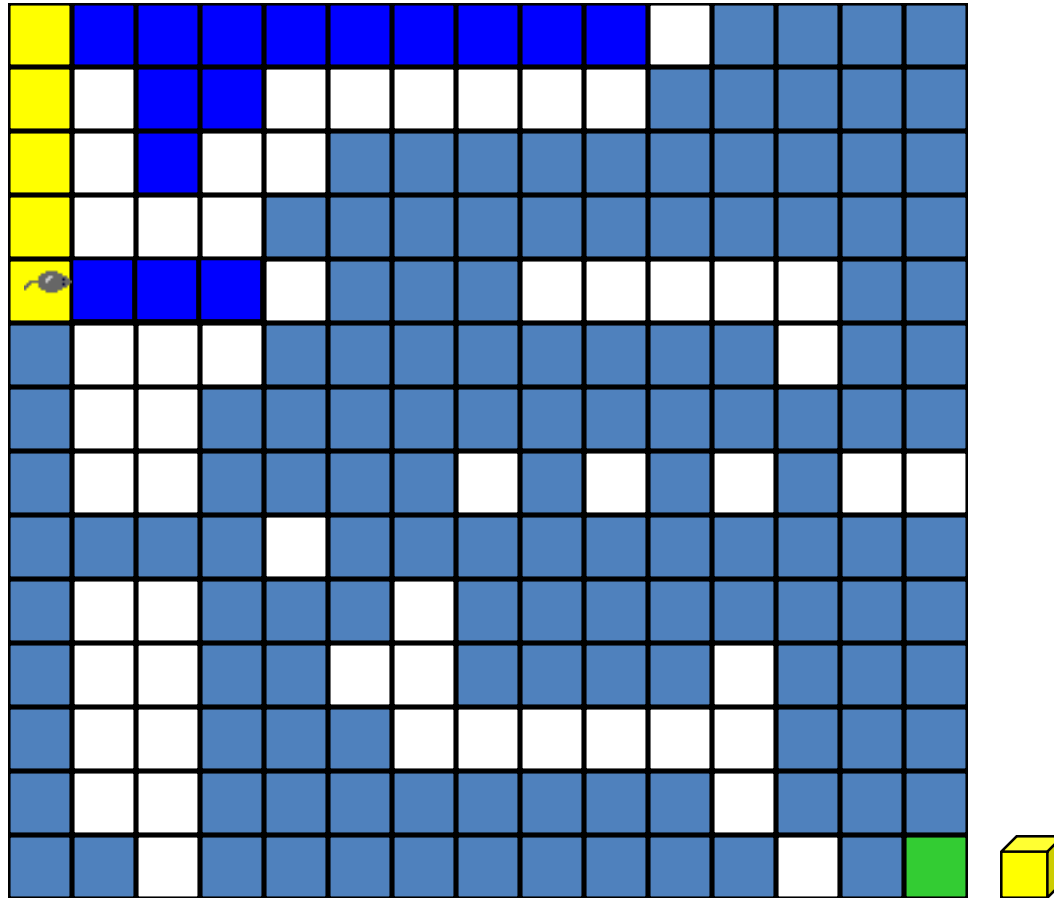
- Move backward until we reach a square from which a forward move is possible.
- Move down

Rat In A Maze



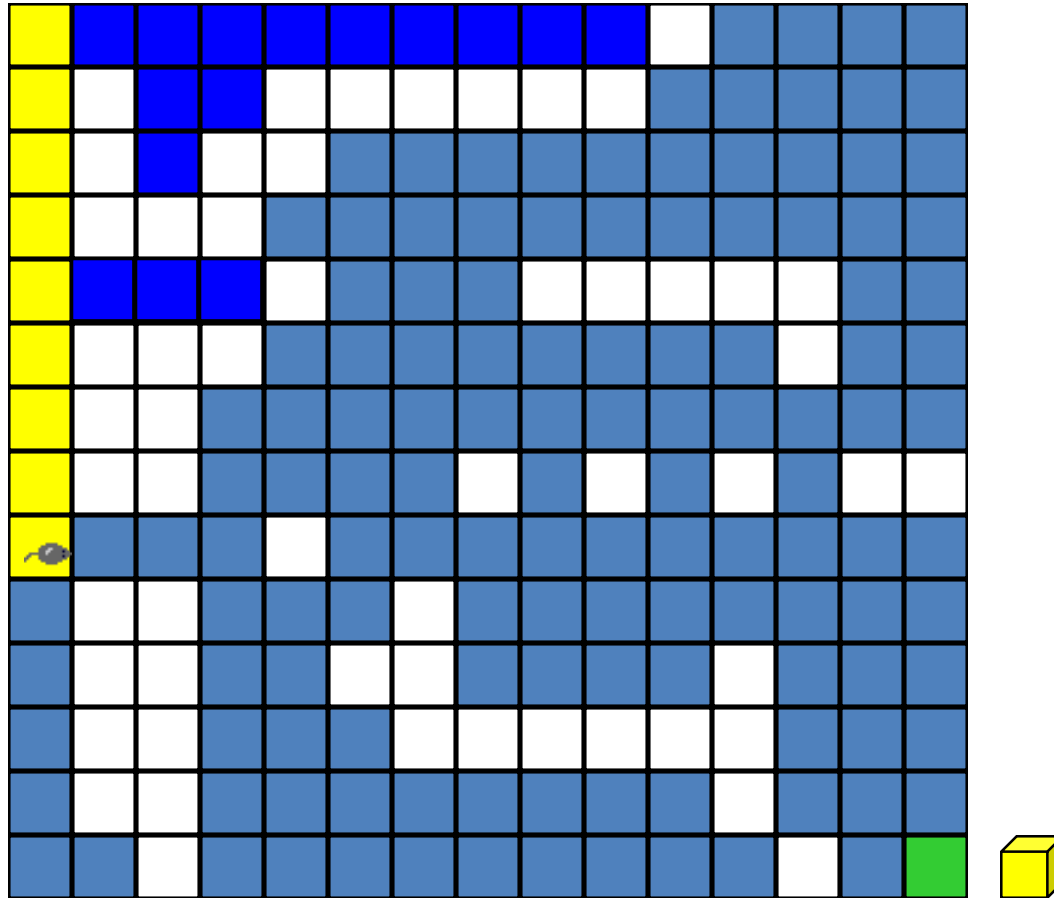
- Move right
- Move Backward

Rat In A Maze



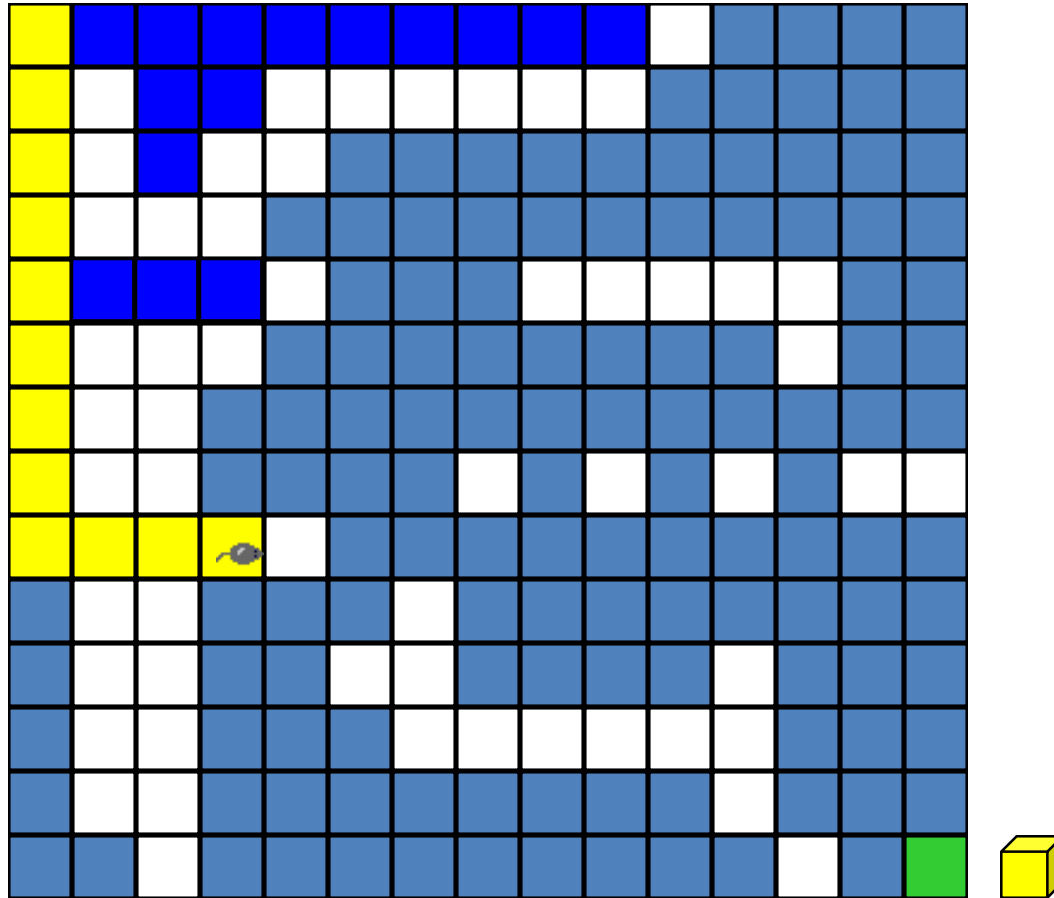
- Move down

Rat In A Maze



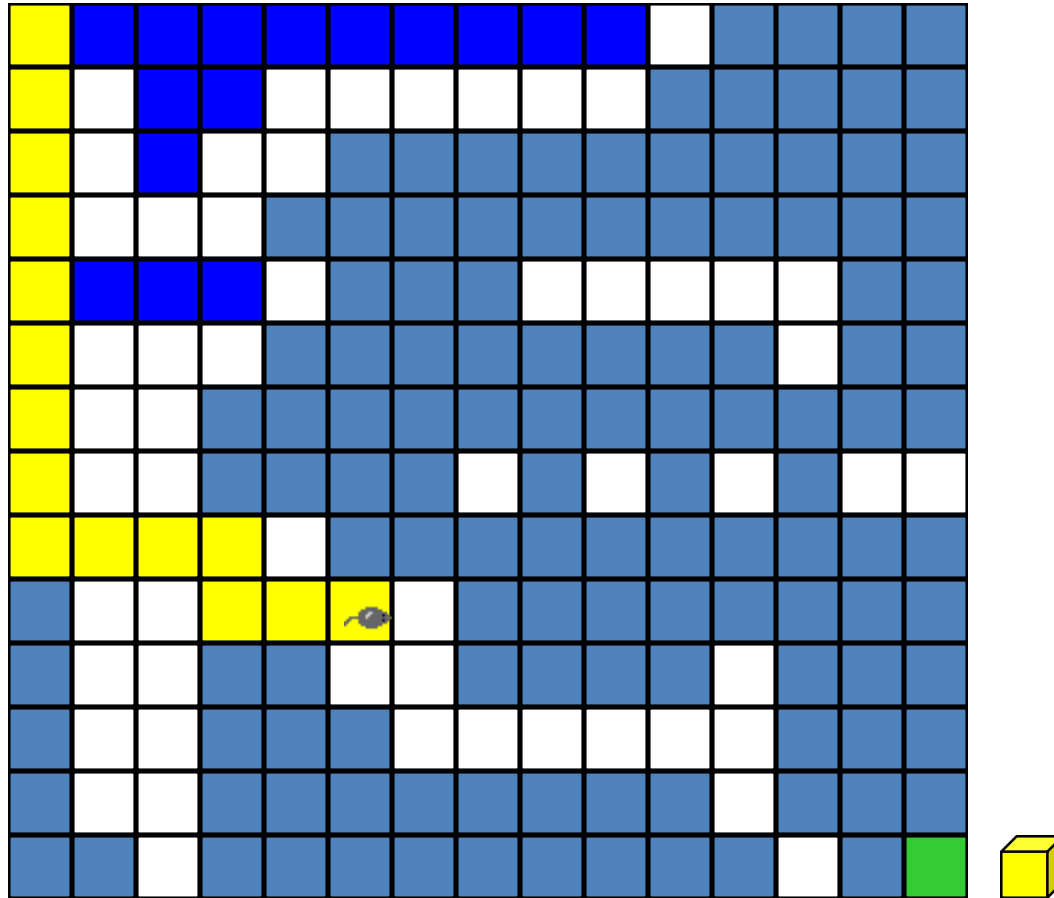
- Move right.

Rat In A Maze



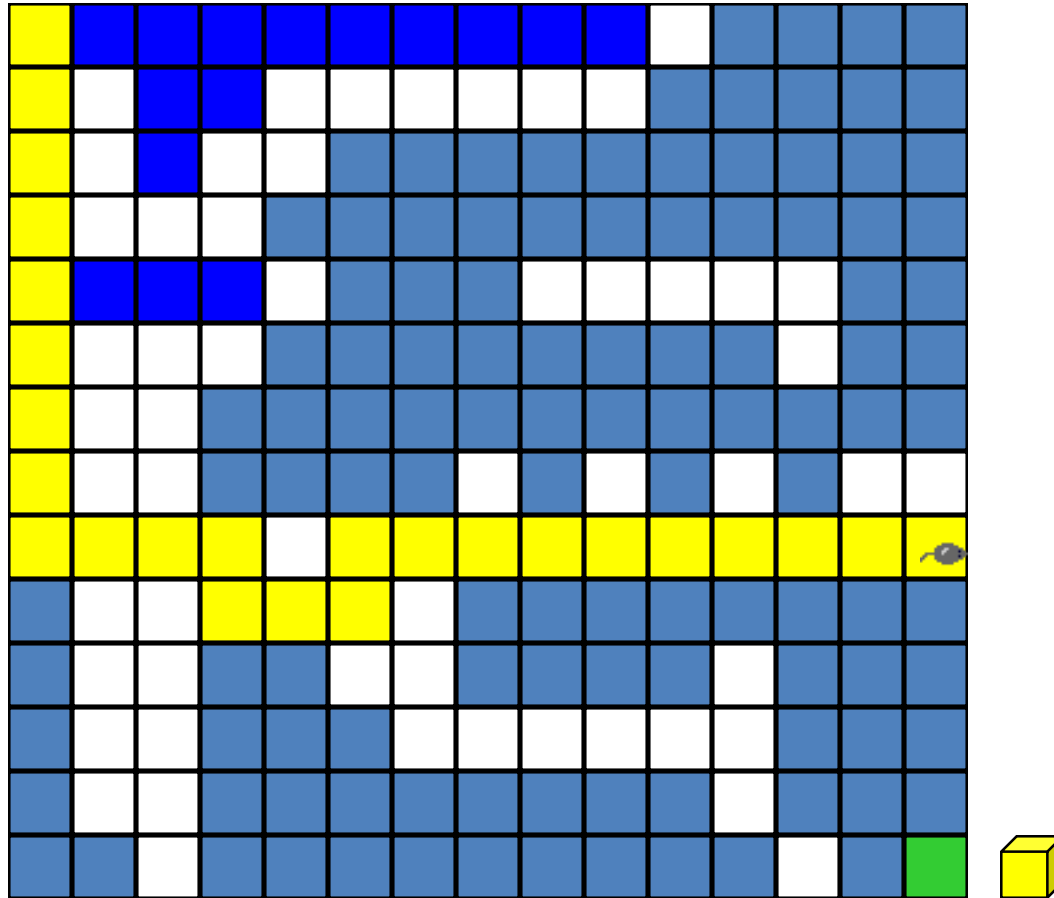
- Move down and then right.

Rat In A Maze



- Move up and then right.

Rat In A Maze



- Move down to exit and eat cheese.

The rate's strategy

- Move forward whenever possible: Not blocked && not visited
 - Remember the footprints
- Move back
 - Try next possible move
- Question: What's the suitable data structure for path storage?
 - stack!

To do in computer: A Mazing Problem

Problem: find a path from the entrance to the exit of a maze.

Representing a maze:

- $\text{maze}[i][j]$, $1 \leq i \leq m$, $1 \leq j \leq p$.
 - ✓ 1 --- blocked, 0 --- open.
 - ✓ To avoid checking for border conditions, the maze is surround by a boarder of ones. So the array will therefore be declared as $\text{maze}[m+2][p+2]$.
 - ✓ The entrance --- $\text{maze}[1][1]$
 - ✓ The exit --- $\text{maze}[m][p]$

entrance

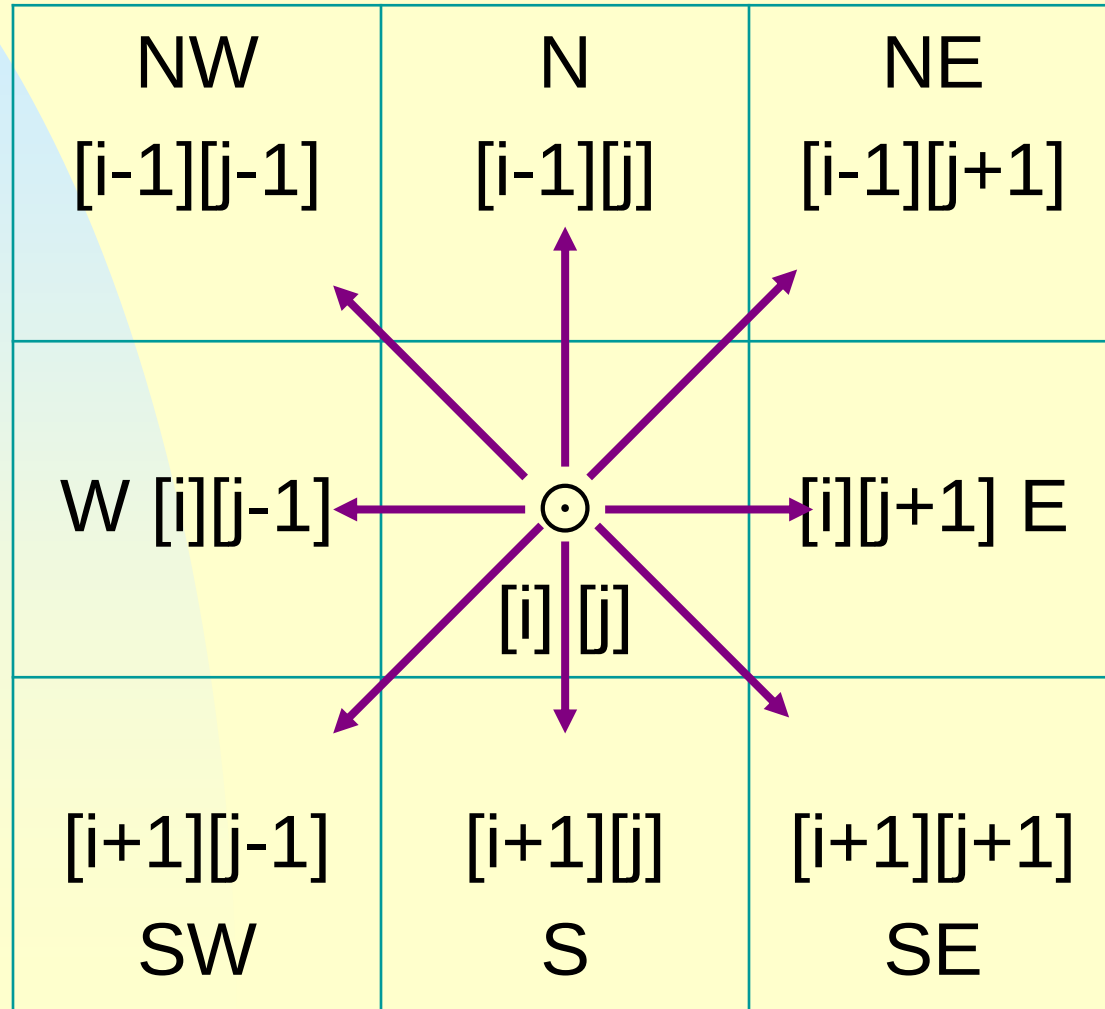
0	1	0	0	1	1	0	1	1
1	0	0	1	0	0	1	1	1
0	1	1	0	1	1	1	0	1
1	1	0	0	1	0	0	1	0
1	0	0	1	0	1	1	0	1
0	0	1	1	0	1	0	1	1
0	1	0	0	1	1	0	0	0

exit

An example maze

Representing the moving action:

- **assume the current point:** $[i][j]$.
- **8 possible moving directions:** N, NE, E, SE, S, SW, W and NW.



- So we predefine **an array(table), move**, to express the **moving action**.

- The data types are:

- ✓ **offsets** `move[8];`

- ✓ **struct** `offsets {int a, b};`

//a and b are the offsets for row and column

- ✓ **enum** `directions`

`{N, NE, E, SE, S, SW, W, NW};`

For example, if we find `[g][h]` from `[i][j]` in **SW** direction, we set:

`g=i+move[SW].a;`

`h=j+move[SW].b;`

q	move [q].a	move [q].b
N	-1	0
NE	-1	1
E	0	1
SE	1	1
S	1	0
SW	1	-1
W	0	-1
NW	-1	-1

The basic idea:

- In the current position $[i][j]$, if we have several directions to go, we **pick one direction d** and get the new position $[g][h]$.
- If $[g][h]$ is **the goal**, success.
- If $[g][h]$ is **an accessible position**, save $[i][j]$ and **$d+1$** in a **stack**, and make $[g][h]$ be the new current position.
- If $[g][h]$ is **not an accessible position**, try another direction. If all the directions are tried and failed, top and pop an element.
- Repeat until either success or every possibility is tried.

- In order to **prevent us from going down the same path twice**:
 - ✓ use another array **mark[m+2][p+2]**, which is initially 0.
 - ✓ **mark[i][j]** is set to 1 once the position is visited.

Algorithm for finding a path through a maze

Initialize stack to the maze entrance coordinates and direction east;

while (stack is not empty)

{

(i, j, dir)=coordinates and direction from top of stack;

pop the stack;

while (there are more moves from (i, j))

{

(g, h)= coordinates of next move;

if ((g==m) && (h==p)) success; //output the path


```
    if ((!maze[g][h]) && (!mark[g][h])) // accessible and not visited
    {
        mark[g][h]=1;
        dir=next direction to try;
        push (i, j, dir) to stack;
        (i, j, dir) = (g, h, N);
    }
    else dir=next direction to try;
}
}
cout << "No path in maze."<< endl;
```

To give a precise **path finding function Path** (in the next slide Program 3.16), we assume:

- ✓ The arrays, **maze, mark, and move** are **global** to **Path**.
- ✓ **stack** is a **stack of items** which is defined as:

```
struct Items {  
    int x, y, dir;  
};
```
- ✓ The struct **Items** has a constructor (not shown) that sets the values of **x, y** and **dir**.
- ✓ The operator **<<** is **overloaded** for both **class Stack** and **struct Items**. It is also declared as **the friend function** of **Stack** to access the private data numbers of **Stack**.
- ✓ To avoid doubling capacity during stack pushing, we can **initial the size of stack to $m \times p$** .

Program 3.16: Finding a path through a maze

void Path(const int m, const int p)

{ //Output a path (if any) in the maze; maze has been initialed that

// maze[0][i] = maze[m+1][i]

// = maze[j][0] = maze[j][p+1] = 1, $0 \leq i \leq p+1$, $0 \leq j \leq m+1$.

// start at (1,1)

mark[1][1]=1;

Stack<Items> stack(m*p);

Items temp(1, 1, E);

stack.Push(temp);

while (!stack.IsEmpty())

{

temp= stack.Top();

stack.Pop(); //unstack an element to try.

int i=temp.x; **int** j=temp.y; **int** d=temp.dir;

```
while (d<8) //eight directions
{
    int g=i+move[d].a; int h=j+move[d].b;
    if ((g==m) && (h==p)) { // reached exit
        // output path
        cout <<stack; //<< is overloaded
        cout << i<<" ," << j<<" ,"<<d<< endl; //last two squares
        cout << m<<" ,"<< p<< endl;          // on the path
        return;
    } //end of if
}
```

```

if ((!maze[g][h]) && (!mark[g][h])) { //can enter new position
    mark[g][h]=1;
    temp.x=i; temp.y=j; temp.dir=d+1;
    stack.Push(temp); // stack it
    i=g ; j=h ; d=N; // move to (g, h) to begin a new try
}

```

```

    else d++; // try next direction

```

```

} //end of the inner while

```

```

} //end of outer while

```

```

cout << "No path in maze." << endl;

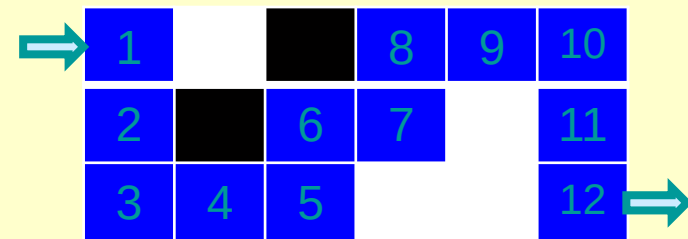
```

```

}

```

An example



Dir Order: N, E, S, W

White: accessible

Black: blocked

Blue: the path
(initially white)

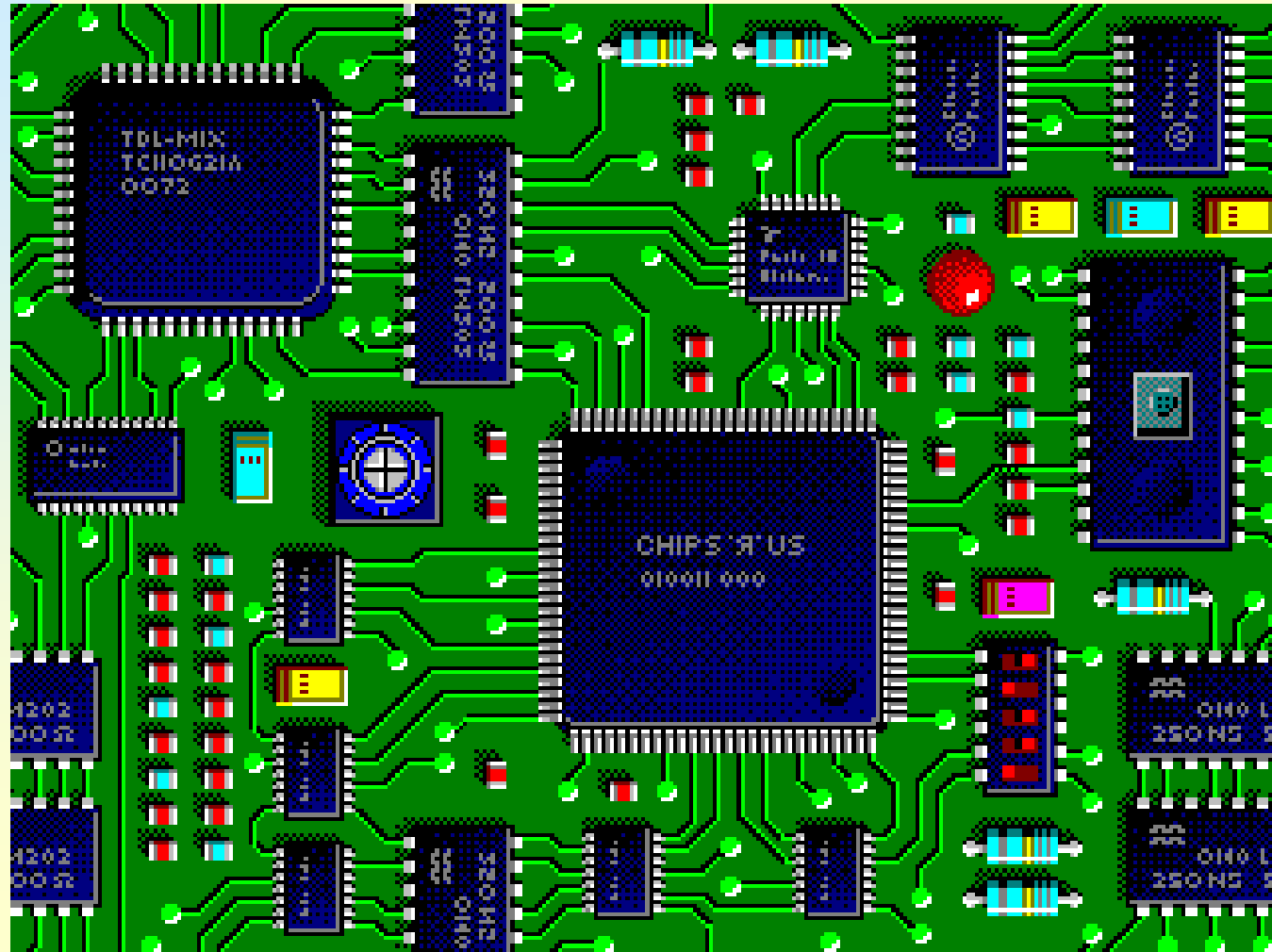
Analysis of Path: No position is visited twice and the time for each marked position is $O(1)$, so the computing time is $O(m*p)$.

The codes of **overloading operator <<**:

```
template <class T>
ostream& operator<<(ostream& os, Stack<T>& s)
{
    os << "top=" << s.top << endl;
    for (int i=0; i<=s.top; i++);
        os<<i<<"."<<s.stack[i]<< endl;
    return os;
}
```

```
ostream& operator<<(ostream& os, Items& item)
{
    return os<<item.x<<"."<<item.y<<"."<<item.dir-1;
    // Note: item.dir is the next direction to go, so the current
    // direction is item.dir-1. Notice the error in the textbook.
}
```

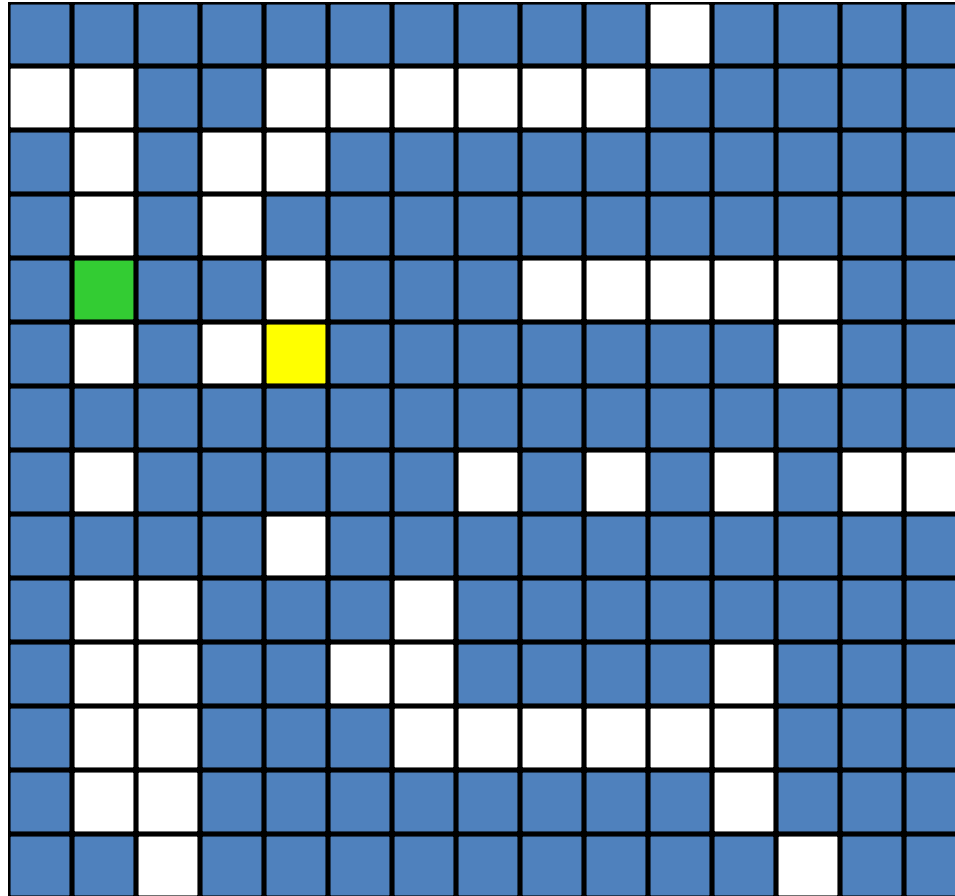
Wire Routing



Lee's Wire Router

 start pin

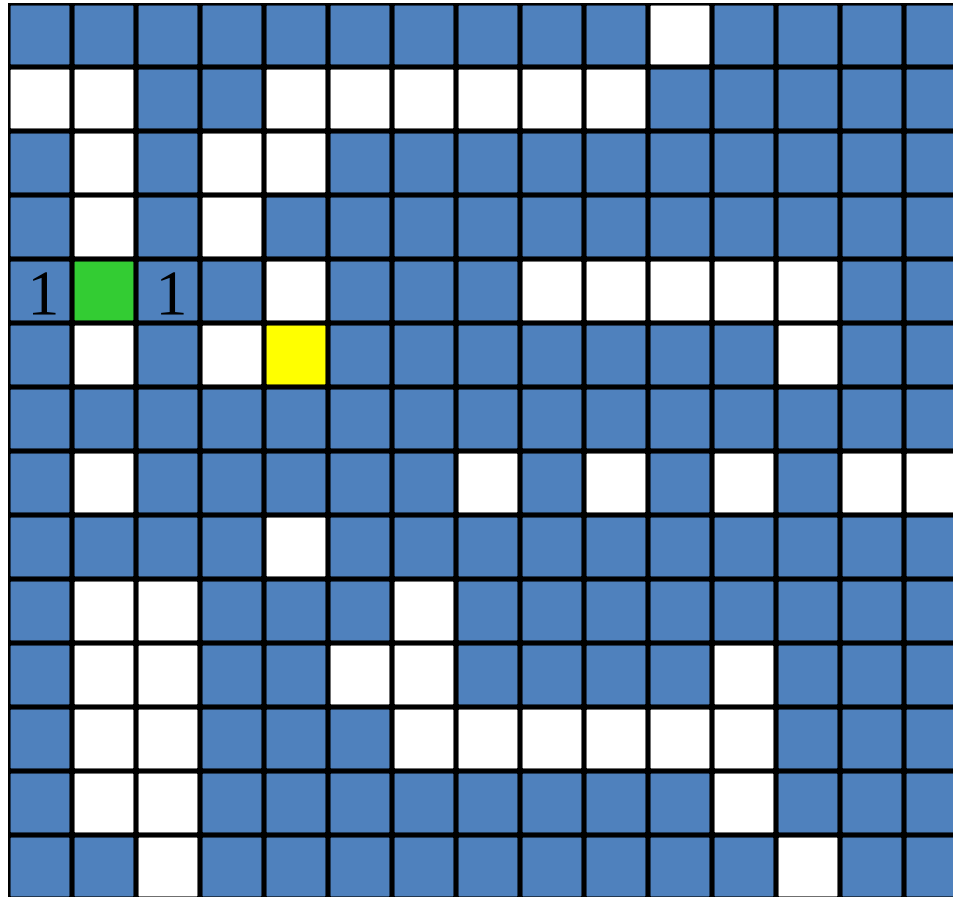
 end pin



Label all reachable squares **1 unit** from start.

Lee's Wire Router

 start pin
 end pin

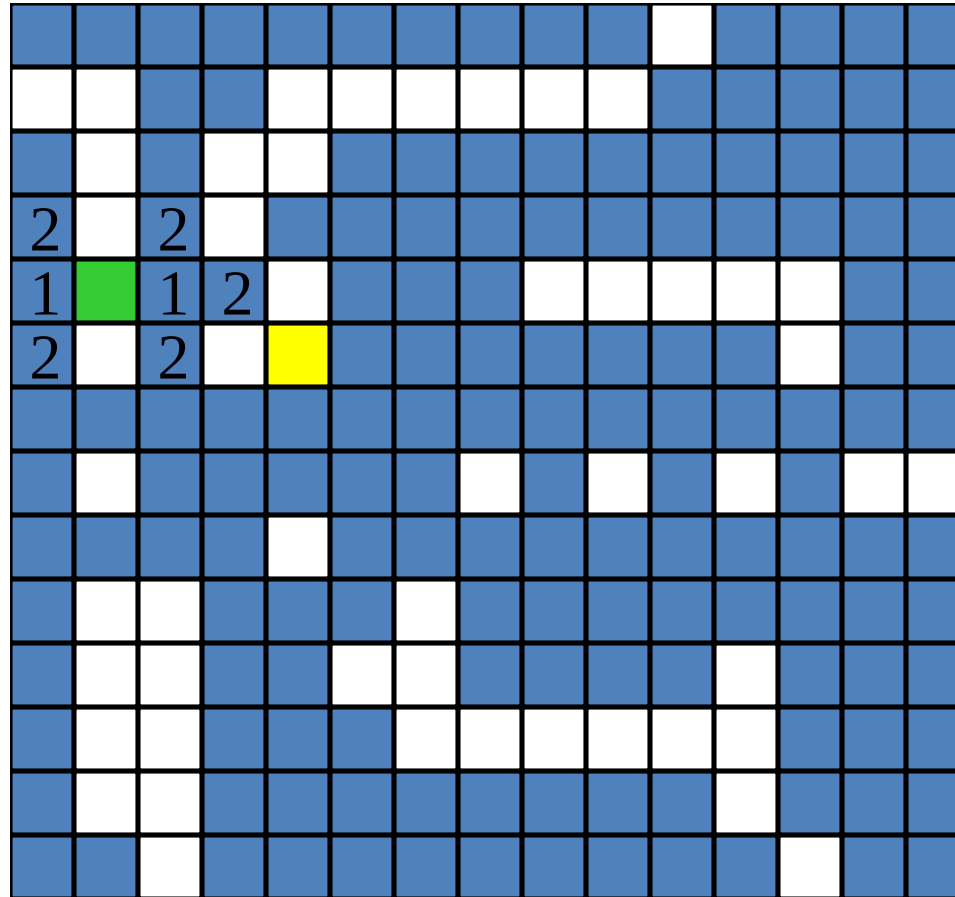


Label all reachable unlabeled squares **2 units** from start.

Lee's Wire Router

 start pin

 end pin

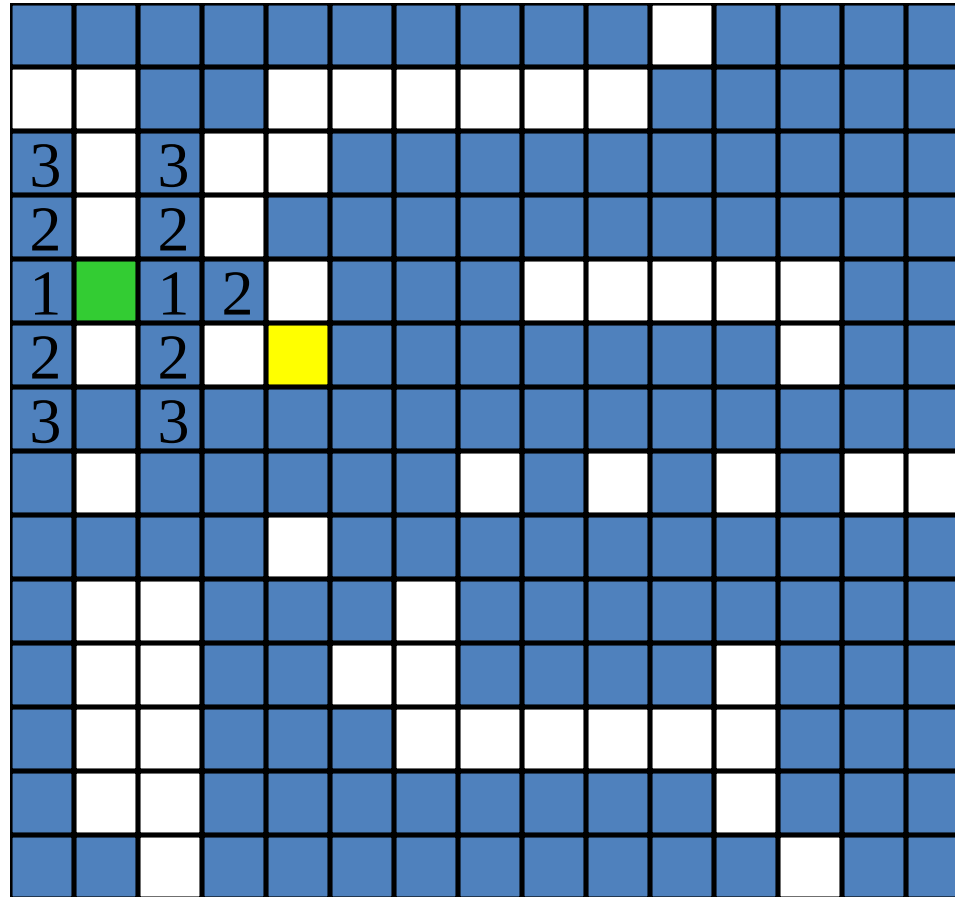


Label all reachable unlabeled squares **3 units** from start.

Lee's Wire Router

 start pin

 end pin

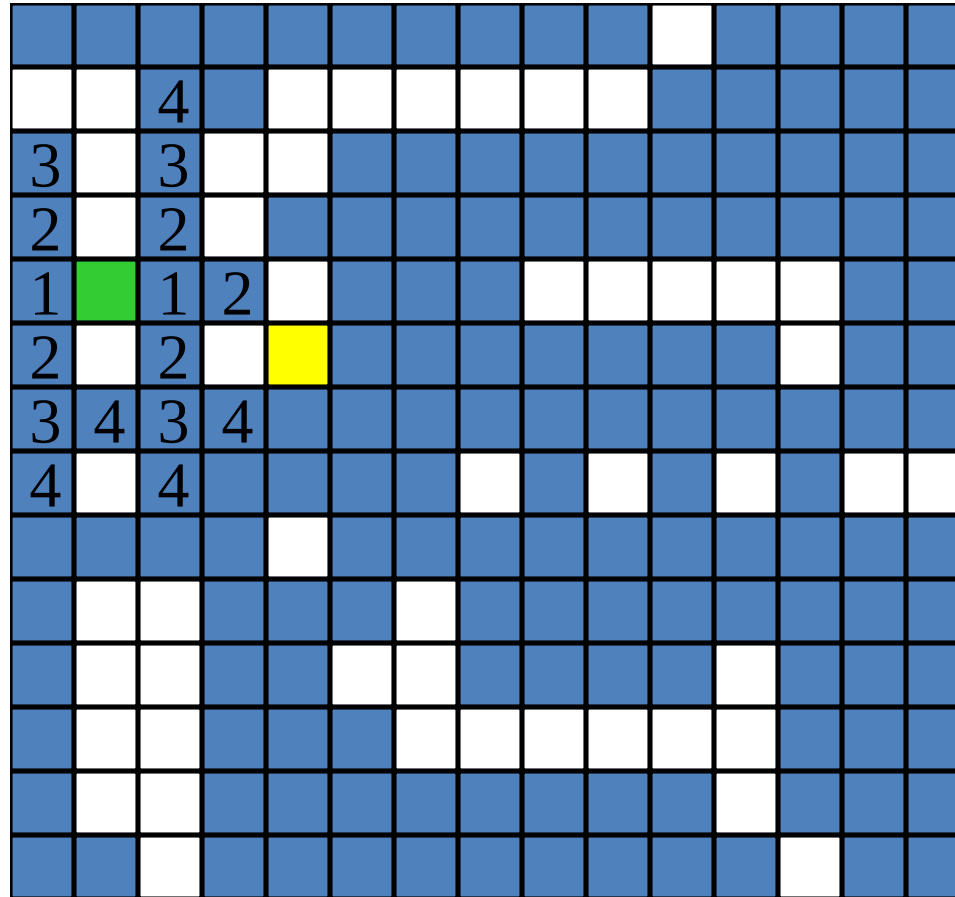


Label all reachable unlabeled squares **4 units** from start.

Lee's Wire Router

 start pin

 end pin

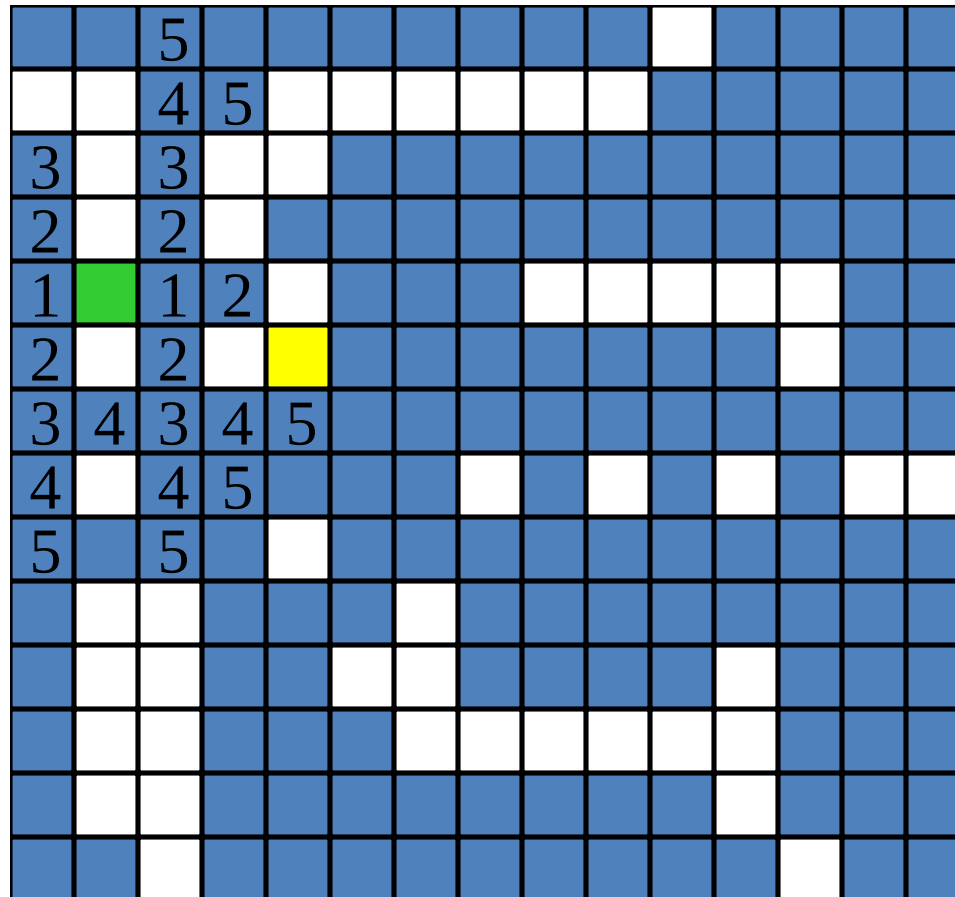


Label all reachable unlabeled squares **5 units** from start.

Lee's Wire Router

 start pin

 end pin

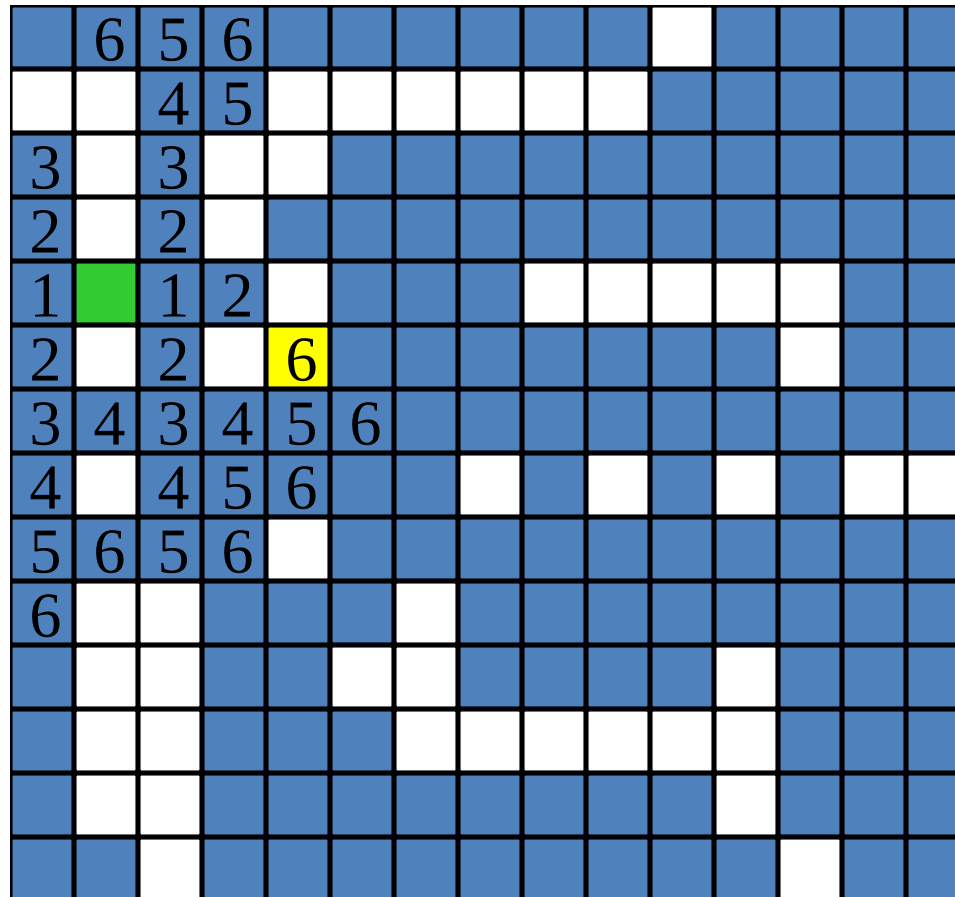


Label all reachable unlabeled squares **6 units** from start.

Lee's Wire Router

 start pin

 end pin



End pin reached. Traceback.

Basic idea----using a queue:

```
Q.Push(startPin);
while(Q.isNotEmpty())
{
    Pin  $p$  = Q.Front(); Q.Pop();
    For all neighbours  $pi$  of  $p$ 
    {
        if (pi==endPin) { //success
            Traceback; //come back from the pin with the
                //distance  $n$  to the only one with the distance to  $n-1$ 
            return; }
        if (pi is not visited && pi is reachable){
            Visit(pi) //record the distance from start pin
            Q.Push(pi); }
    }
}
return; //failure
```

3.6 Evaluation of expressions

- Expressions comprise three kinds of entities:
 - Operators
 - Operands
 - Delimiters
- Infix Form
 - Normal way to write an expression.
 - Binary operators come in between their left and right operands.

Operator Priorities

To fix the order of evaluation, we assign to each operator a priority.

priority	operator	
1	unary minus, !	highest
2	*, /, %	
3	+, -	
4	<, <=, >, >=	
5	==, !=	
6	&&	
7		lowest

Some rules for infix Expressions

- (1) Within any pair of parentheses, operators with the highest priority will be evaluated first.
- (2) Evaluation of operators of the same priority will proceed left to right.
- (3) By using parentheses we can override these rules, as expressions are always evaluated with the innermost parenthesized first.
- (4) When an operand lies between two operators, the operand associates with the operator that has higher priority.
- (5) When an operand lies between two operators that have the same priority, the operand associates with the operator on the left.
- (6) Subexpression within delimiters is treated as a single operand, independent of the remainder of the expression.

Problems of Infix Expression

- Need operator priorities, rules for breaking tie, and delimiters.
 - ✓ This makes computer difficult to evaluate an expression with the infix form.
- Postfix and prefix expression forms do not rely on operator priorities, rules for breaking tie, or delimiters.
 - ✓ So it is easier for a computer to evaluate expressions that are in these forms.

Postfix Form

■ Characteristics:

- ✓ The postfix form of a variable or constant is the same as itself.
- ✓ An operator appears immediately after its operands.
- ✓ The relative order of operands is the same in infix and postfix forms.
- ✓ Unary operators is replaced with new symbols.

■ Advantages:

- ✓ There is no parentheses
- ✓ If we evaluate the expression, the priority of the operators is no longer relevant

Postfix Examples

- Infix = $a + b * c$

Postfix = $abc*+$

- Infix = $a * b + c$

Postfix = $ab*c+$

- Infix = $(a + b) * (c - d) / (e + f)$

■ Postfix = $ab+cd-*ef+ /$

- Infix = $- a - b$

Postfix = $a?b-$

Postfix evaluation

■ Basic idea:

make a **left to right** scan :

✓ if the scanned token is an operand, we stack it.

✓ if the scanned token is an operator, we

evaluate operators using the correct number of operands from the stack and places the result onto the stack.

Postfix evaluation example

postfix: $A\ B\ /\ C\ -\ D\ E\ *\ +\ A\ C\ *\ -$

The corresponding infix: $A\ /\ B\ -\ C\ +\ D\ *\ E\ -\ A\ *\ C$

The remaining postfix	stack	operation
$B\ /\ C\ -\ D\ E\ *\ +\ A\ C\ *\ -$	A	
$\ /\ C\ -\ D\ E\ *\ +\ A\ C\ *\ -$	AB	
$C\ -\ D\ E\ *\ +\ A\ C\ *\ -$	T_1	$T_1 = A/B$
$\ -\ D\ E\ *\ +\ A\ C\ *\ -$	T_1C	
$D\ E\ *\ +\ A\ C\ *\ -$	T_2	$T_2 = T_1 - C$
$E\ *\ +\ A\ C\ *\ -$	T_2D	
$\ *\ +\ A\ C\ *\ -$	T_2DE	

postfix: A B / C – D E * + A C * –

The remaining postfix	stack	operation
+ A C * –	$T_2 T_3$	$T_3 = D * E$
A C * –	T_4	$T_4 = T_2 + T_3$
C * –	$T_4 A$	
* –	$T_4 AC$	
–	$T_4 T_5$	$T_5 = A * C$
	T_6	$T_6 = T_4 - T_5$

T_6 is the result.

Program 3.18: Evaluating postfix expressions

```
void Eval(Expression e)
{ // evaluate the postfix expression e. It is assumed that the
  // last token in e is '#'. A function NextToken is used to get
  // the next token from e.
  Stack<Token> stack; //initialize stack
  for (Token x = NextToken(e); x!='#'; x=NextToken(e))
    if (x is an operand) stack.Push(x);
    else { // operator
      remove the correct number of operands for operator x
      from stack; perform the operation x and store the result
      (if any) onto the stack;
    }
}
```

3.6.3 Infix to Postfix

- We establish two priorities for **operators**:
 - **isp** (in-stack priority)
 - **icp** (in-coming priority)
- The **isp** and **icp** of the operators in the following figure (Figure 3.15, P160) are the same.

<u>priority</u>	<u>operator</u>
1	unary minus, !
2	*, /, %
3	+, -
4	<, <=, >, >=
5	==, !=
6	&&
7	

Note that: **the smaller number has the higher priority.**

■ View ‘(’ and ‘#’ as operators:

- $\text{icp}('()=0$, $\text{isp}('()=8$, $\text{isp}('#)=8$ (‘#’ is the last token in an expression),
- ‘)’ makes **unstacking** until ‘(’, and **never be stacked**.
- We can see **the left parenthesis**:
 - when it is **not in the stack**, it behaves as an operator **with high priority**,
 - whereas **once it get in**, it behaves as one **with low priority** (no operator other than the matching right parenthesis should cause it to get unstacked).

A priority-based idea:

(1) **Stack** “#”;

(2) Make a left to right scan:

- ✓ If the scanned token is an **operand**, immediately output it.
- ✓ If the scanned token is an **operator** (including “(” and “#”),
 - Operators are taken out of stack as long as their **isp** is **numerically less than or equal to** the **icp** of this new operator.
 - This new operator gets in the stack if its **icp** is **numerically less than** those in the stack.
- ✓ If the scanned token is “)”, unstack until “(”.

Example:

A*(B+C)*D#

→ABC+*D*#

next token	stack	output
A	#	A
*	#*	A
(	#*(	A
B	#*(	AB
+	#*(+	AB
C	#*(+	ABC
)	#*	ABC+
*	#*	ABC+ *
D	#*	ABC+ *D
#	#	ABC+ *D*

') ' makes unstacking until '(' . '(' doesn't output.

The 2nd *

The 1st *

Program 3.19: Converting from infix to postfix form

void **Postfix** (Expression e)

```
{ // output the postfix of the infix expression e. It is assumed  
  // that the last token in e is '#'. Also, '#' is used at the bottom  
  // of the stack.
```

```
    Stack<Token> stack; //initialize stack
```

```
    stack.Push('#');
```

```

for (Token x=NextToken(e); x!='#'; x=NextToken(e))
    if (x is an operand) cout<<x;
    else if (x=='(')
        { // unstack until '('
            for (; stackTop()!='('; stack.Pop())
                cout<<stack.Top();
            stack.Pop(); // unstack '('
        }
    else { // x is an operator
        for (; isp(stack.Top()) <= icp(x); stack.Pop())
            cout<<stack.Top();
        stack.Push(x);
    }
// end of expression, empty the stack
for (; !stack.IsEmpty()); cout<<stack.Top(), stack.Pop());
cout << endl;
}

```

Analysis of Postfix:

- The function makes **only a left-to-right pass** across the input. The time spent on **each operand is $O(1)$** .
- Each operator is stacked and unstacked **at most once**. Hence, the time spent on **each operator is also $O(1)$** .
- So the complexity of Postfix is $\Theta(n)$.
- The stack will **not be deeper than 1 ('#') + the number of operators** in e.

Exercise(s) and Experiment(s)

Exercise(s):

- **P138:1; P147:3; P157:3; P165:1,2,3(a)**

Experiment(s):

- **P168:1,2**